

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Procedurální generování dungeonů ve 2D hrách

Implementace a srovnání algoritmů v herním enginu Godot



2026

Vedoucí práce:
doc. RNDr. Jan Konečný, Ph.D.

Jesse Sadowý

Studijní program: Informační technologie,
kombinovaná forma

Bibliografické údaje

Autor: Jesse Sadowý

Název práce: Procedurální generování dungeonů ve 2D hrách (Implementace a srovnání algoritmů v herním enginu Godot)

Typ práce: bakalářská práce

Pracoviště: Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci

Rok obhajoby: 2026

Studijní program: Informační technologie, kombinovaná forma

Vedoucí práce: doc. RNDr. Jan Konečný, Ph.D.

Počet stran: 57

Přílohy: zdrojový kód aplikace

Jazyk práce: český

Bibliographic info

Author: Jesse Sadowý

Title: Procedural Generation of Dungeons in 2D Games (Implementation and Comparison of Algorithms in Godot Game Engine)

Thesis type: bachelor thesis

Department: Department of Computer Science, Faculty of Science, Palacký University Olomouc

Year of defense: 2026

Study program: Information Technologies, combined form

Supervisor: doc. RNDr. Jan Konečný, Ph.D.

Page count: 57

Supplements: application source code

Thesis language: Czech

Anotace

Bakalářská práce se zabývá procedurálním generováním dungeonových map ve 2D hrách. Jsou popsány základní principy procedurálního generování obsahu a vybrané metody tvorby dungeonových struktur. V herním engineu Godot bylo implementováno a experimentálně porovnáno pět generovacích algoritmů z hlediska času generování a pokrytí mapy průchozím prostorem. Výsledky ukazují výrazné rozdíly mezi metodami a práce diskutuje vhodnost jednotlivých přístupů pro různé typy herního prostředí.

Synopsis

This bachelor's thesis deals with procedural generation of dungeon maps in 2D games. It describes the basic principles of procedural content generation and selected methods for creating dungeon structures. Five generation algorithms were implemented and experimentally compared in the Godot game engine in terms of generation time and floor coverage. The results show significant differences between the methods, and the thesis discusses the suitability of each approach for different types of game environments.

Klíčová slova: procedurální generování; dungeon; 2D hry; Godot; herní engine; algoritmus

Keywords: procedural generation; dungeon; 2D games; Godot; game engine; algorithm

Děkuji vedoucímu práce doc. RNDr. Janu Konečnému, Ph.D. za odborné vedení a cenné připomínky při vypracování této práce.

Odevzdáním tohoto textu jeho autor/ka místopřísežně prohlašuje, že celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

Obsah

1	Úvod	8
2	Současný stav řešené problematiky	8
2.1	Procedurální generování obsahu ve hrách	8
2.1.1	Vymezení a definice pojmu	8
2.1.2	Nevýhody procedurálního generování	9
2.1.3	Přístupy k procedurálnímu generování	9
2.1.4	Využití procedurálního generování v herním průmyslu	10
2.2	Roguelike hry a jejich vztah k procedurálnímu generování	10
2.2.1	Beneath Apple Manor	11
2.2.2	Rogue	11
2.2.3	Elite	12
2.3	Generování dungeonů ve 2D hrách	13
2.3.1	Požadavky na generování dungeonových map	13
2.4	Přehled metod generování	14
2.4.1	Náhodné umístění místností a chodeb	14
2.4.2	Binary Space Partitioning	15
2.4.3	Metoda náhodné procházky	16
2.4.4	Buněčné automaty	16
2.4.5	Metoda bludiště	17
2.4.6	Další metody	17
2.5	Shrnutí způsobů generování	18
3	Experimentální část	19
3.1	Použité technologie	19
3.1.1	Herní engine Godot	19
3.2	Návrh systému generování	20
3.2.1	Reprezentace mapy	20
3.2.2	Struktura generátoru	20
3.2.3	Vizualizace mapy a hráče	21
3.2.4	Ovládání generování a zobrazení statistik	22
3.3	Implementace generovacích metod	23
3.3.1	Náhodné generování místností a chodeb	23
3.3.2	Binary Space Partitioning	24
3.3.3	Metoda náhodné procházky	25
3.3.4	Buněčné automaty	26
3.3.5	Metoda bludiště	27
3.4	Metodika měření a vyhodnocení	28
4	Výsledky	29
4.1	Výsledky jednotlivých algoritmů	29
4.1.1	Rooms (místnosti a chodby)	29
4.1.2	BSP (Binary Space Partitioning)	30

4.1.3	Drunken Walk (náhodná procházka)	32
4.1.4	Cellular (buněčné automaty)	33
4.1.5	Maze (bludiště)	34
4.2	Srovnání algoritmů	35
4.2.1	Průměrný čas generování	36
4.2.2	Průměrné pokrytí	36
4.2.3	Škálování s velikostí mapy	37
4.2.4	Kompromis mezi rychlostí a pokrytím	38
4.2.5	Rozptyl výsledků na velké mapě	39
4.3	Souhrnná tabulka výsledků	40
4.4	Shrnutí výsledků	40
5	Diskuse	41
5.1	Silné a slabé stránky jednotlivých metod	41
5.2	Zkušenosti z implementace	41
5.3	Souvislost s předchozím výzkumem	42
5.4	Možnosti dalšího rozšíření práce	42
	Závěr	43
	Conclusions	44
A	Ukázky zdrojového kódu	45
A.1	Generátor místností a chodeb	45
A.2	Generátor BSP	47
A.3	Generátor náhodné procházky	49
A.4	Generátor buněčných automatů	50
A.5	Generátor bludiště	52
B	Odkaz na repozitář projektu	55
C	Zkratky	55
D	Obsah elektronických dat	55
	Seznam zkratk	56
	Literatura	57

Seznam tabulek

1	Srovnání vybraných metod generování dungeonových map	18
2	Ukázka reprezentace mapy v mřížce.	20
3	Souhrnné výsledky měření – průměrný čas generování a pokrytí .	40

1 Úvod

Procedurální generování obsahu představuje v současném herním vývoji významný přístup, který umožňuje automatizovaně vytvářet části herního světa na základě předem definovaných pravidel a algoritmů. V oblasti digitálních her se tento princip uplatňuje zejména tam, kde je žádoucí vysoká variabilita prostředí, znovuhratelnost nebo snížení nároků na ruční tvorbu rozsáhlého množství obsahu. Jednou z typických oblastí využití je generování dungeonových map, které tvoří důležitou součást 2D her, zejména titulů inspirovaných žánrem roguelike.

Dungeonové mapy kladou na generování specifické požadavky. Nestačí, aby byly pouze náhodné nebo vizuálně rozmanité, ale musí zároveň splňovat základní podmínky hratelnosti a vhodné rozmístění jednotlivých prvků prostředí. Volba generovací metody proto výrazně ovlivňuje nejen výsledný vzhled mapy, ale i herní zážitek hráče.

Cílem této bakalářské práce je představit vybrané metody procedurálního generování dungeonových map ve 2D hrách, analyzovat jejich vlastnosti a implementovat jejich praktické ukázky v herním engineu Godot. Součástí práce je návrh systému v podobě jednoduché hry, který umožní jednotlivé algoritmy realizovat v jednotném prostředí, vizualizovat jejich výstupy a následně je mezi sebou porovnat a ověřit specifické aspekty důležité právě pro generovaný obsah. Důraz je kladen jak na teoretické vymezení problematiky, tak na praktickou implementaci a zhodnocení výsledků.

Práce je rozdělena do několika kapitol. Nejprve jsou popsány základní principy procedurálního generování, jeho přínosy, omezení a historický vývoj. Další kapitola se věnuje samotnému generování dungeonů ve 2D hrách, požadavkům na výsledné mapy a přehledu vybraných metod. Poté je představen návrh prototypu aplikace a použité technologie. V dalších částech práce je popsána implementace jednotlivých algoritmů, jejich vyhodnocení a vzájemné porovnání. Závěr práce shrnuje dosažené výsledky, omezení a možné rozšíření.

2 Současný stav řešené problematiky

2.1 Procedurální generování obsahu ve hrách

2.1.1 Vymezení a definice pojmu

Procedurální generování ve hrách lze vymezit jako algoritmickou tvorbu herního obsahu s omezeným nebo nepřímým zásahem člověka. Tento přístup umožňuje automatizovaně vytvářet různé prvky hry bez nutnosti jejich ruční přípravy. Herní obsah přitom zahrnuje široké spektrum složek, například mapy, úrovně, pravidla, úkoly, zbraně či předměty, případně i samotné vlastnosti jednotlivých předmětů. PCG se tedy neomezuje pouze na herní prostředí, ale může být aplikováno i na další části herního systému. V závislosti na konkrétním návrhu může ovlivňovat nejen prostorovou strukturu hry, ale také její mechaniky, obtížnost či

samotný průběh hraní [1].

V současnosti je pojem generování obsahu často spojován s metodami umělé inteligence, tato práce se však zaměřuje výhradně na algoritmické postupy založené na předem definovaných pravidlech. Nezabývá se tedy adaptivním ani učícím se chováním systémů umělé inteligence.

Pojem hra je v odborné literatuře obtížné jednoznačně vymezit, protože zahrnuje široké spektrum forem lišících se cíli, pravidly, mírou interaktivity i použitou platformou. V kontextu této práce je proto termín hra chápán jako digitální videohra realizovaná v reálném čase, ve které hráč interaguje s virtuálním prostředím přes definované herní mechaniky. Pro účely dalšího textu se práce zaměřuje především na 2D videohry určené pro osobní počítače, případně další běžné platformy.

Termíny „procedurální“ a „generování“ zdůrazňují, že jde o proces realizovaný pomocí počítačových procedur a algoritmů. Člověk nastavuje vstupní podmínky a spouští samotný proces, výsledkem je však automaticky vytvořený herní obsah, který není pevně předem daný, ale vzniká až za běhu programu.

2.1.2 Nevýhody procedurálního generování

Přestože procedurální generování obsahu přináší řadu výhod, jeho využití s sebou nese také několik významných omezení a úskalí.

Jedním z hlavních problémů je nižší míra kontroly nad výsledným obsahem. U ručně navržených úrovní má vývojář plnou kontrolu nad tempem hry, rozmístěním výzev, předmětů i klíčových herních momentů. Naproti tomu u procedurálního generování je výsledná podoba světa závislá na implementovaném algoritmu a zvolených parametrech. To může vést k nevyváženým úrovním, nelogickému rozmístění prvků nebo situacím, které jsou příliš jednoduché či naopak extrémně náročné. Vývojář proto musí věnovat značné úsilí návrhu, ladění a testování generátoru, aby bylo dosaženo uspokojivé kvality výsledného obsahu.

Dalším rizikem je možnost vzniku repetitivního nebo uměle působícího obsahu. I přes to, že je obsah generován automaticky a náhodně, často vychází z omezené množiny předem definovaných prvků a pravidel. Pokud není systém navržen dostatečně variabilně, může hráč postupně rozpoznat opakující se vzorce, což může negativně ovlivnit herní zážitek. Tento problém je sice možné minimalizovat vhodným návrhem algoritmu a důkladným laděním, nicméně zcela eliminovat jej bývá obtížné [1].

2.1.3 Přístupy k procedurálnímu generování

Na procedurální generování se můžeme podívat z různých úhlů pohledu a lze ho rozdělit podle určitých specifik, například podle toho, jak systém reaguje na hráče, případně zda využíváme konkrétní vstupní parametry.

Vzhledem k chování hráče se můžeme v praxi nejčastěji setkat s adaptivním či neadaptivním přístupem. V případě neadaptivního (statického) generování je obsah vytvářen bez ohledu na chování hráče a jeho průběh samotnou hrou.

Všechny generované struktury vycházejí ze stejných pravidel a reakce hráče na podněty neovlivní výsledek. Naopak adaptivní generování pracuje s informacemi o hráči. Systém může sledovat úspěšnost hráče, rychlost reakcí, délku průchodu úrovní, využívané herní prvky nebo způsob řešení situací a na základě těchto dat dále upravovat generovaný obsah.

Dalším důležitým rozdělením je deterministický a stochastický přístup. Deterministické generování vychází z pevně daných počátečních hodnot (například seed) a při stejném vstupu vždy produkuje stejný výstup. Tento přístup umožňuje reprodukovatelnost výsledků, což je výhodné například při testování nebo sdílení konkrétních map mezi hráči. Stochastické generování naproti tomu pracuje s náhodností bez pevně stanoveného počátečního stavu, takže výsledky se budou lišit i při opakovaném spuštění algoritmu.

Z hlediska samotného procesu generování lze rozlišit konstruktivní metody a přístup označovaný jako *generate-and-test*. Konstruktivní metody vytvářejí obsah postupně podle předem definovaných pravidel a omezení. Každý krok generování přímo přispívá k vytvoření výsledné struktury. Naproti tomu metoda *generate-and-test* nejprve vytvoří potenciální řešení, které je následně ověřeno podle stanovených kritérií. Pokud výsledek nevyhovuje, proces se opakuje. Tento přístup může být výpočetně náročnější, ale umožňuje lépe kontrolovat výsledné vlastnosti mapy.

V literatuře se rovněž objevuje pojem *mixed-authorship* (smíšené autorství), kdy se na tvorbě obsahu podílí jak algoritmus, tak lidský návrhář. Systém může například generovat návrhy úrovní, které následně autor upravuje, případně může reagovat na částečně ručně vytvořenou strukturu. Tento přístup kombinuje kreativitu člověka s rychlostí automatizovaného generování.

Plně automatická generace naopak probíhá bez zásahu člověka během samotného vytváření obsahu. Algoritmus pracuje pouze na základě vstupních parametrů. Tento způsob je typický zejména pro roguelike hry a další tituly, u nichž je cílem vysoká variabilita jednotlivých průchodů [1].

2.1.4 Využití procedurálního generování v herním průmyslu

S procedurálním generováním se dnes setkáváme téměř v každé moderní videohře, jeho počátky však sahají již do 70. let 20. století. V tomto období byl výpočetní výkon i dostupná operační paměť velmi omezené. Tyto limity výrazně omezovaly množství ručně vytvářeného obsahu, a vývojáři proto hledali způsoby, jak vytvářet variabilní herní prostředí s minimálními nároky na paměť RAM. Právě v tomto kontextu se začaly prosazovat procedurální přístupy [1].

2.2 Roguelike hry a jejich vztah k procedurálnímu generování

V kontextu PCG je nutné zmínit také pojem roguelike, který označuje herní subžánr úzce spojený s náhodně generovaným obsahem. Název žánru je odvozen

od hry Rogue, jež výrazně ovlivnila podobu mnoha pozdějších titulů.

Roguelike hry bývají typické náhodně generovanými úrovněmi, tahovým systémem, permanentní smrtí postavy (permadeath) a zpravidla i vyšší obtížností. Náhodné generování zde neplní pouze technickou funkci, ale představuje zásadní designový prvek, který podporuje průzkum, strategické rozhodování i adaptaci hráče na proměnlivé prostředí. Právě díky těmto vlastnostem se roguelike hry staly jedním z nejvýraznějších příkladů praktického a systematického využití procedurálního generování v herním průmyslu [1].

2.2.1 Beneath Apple Manor

Jednou z prvních her, které implementovaly PCG, byla Beneath Apple Manor z roku 1978. Autorem titulu byl Don Worth a hru vydalo studio The Software Factory. Primárně byla vyvíjena pro počítač Apple II.

Herní prostředí v Beneath Apple Manor bylo generováno algoritmicky při každém novém spuštění hry. Díky tomu nebylo rozložení místností, chodeb ani jednotlivých herních prvků či nepřátel nikdy zcela totožné, a právě díky tomu byla hra přívětivá pro znovuhratelnost. Cílem hry bylo získat zlaté jablko a postup bylo možné ukládat, takže po smrti hráče se načetla uložená úroveň znovu, přičemž hráč přišel o část svých atributů.

Autor hry čerpal inspiraci z programu Dragon Maze, který rovněž využíval náhodné výpočty ke generování bludišť. Tento princip však dále rozvinul a navrhl algoritmus vytvářející funkční a hratelnou strukturu dungeonů složenou z místností propojených chodbami. Hra navíc umožňovala hráči ovlivnit některé parametry generování, například počet místností na jednotlivých patrech, barevnou variantu zobrazení nebo obtížnost. Po vytvoření struktury úrovně algoritmus náhodně rozmístil také nepřátele, poklady a další herní objekty. S rostoucí hloubkou dungeonů se zároveň zvyšovala obtížnost hry, zejména síla nepřátel.

Použitý algoritmický přístup lze zařadit mezi rané příklady způsobu generování založeného na místnostech a chodbách, který se později stal jedním ze základních modelů využívaných v hrách typu roguelike. Beneath Apple Manor tak představuje významný historický příklad využití těchto metod, jehož principy byly dále rozvíjeny v následujících dekáдах [2, 3].

2.2.2 Rogue

Za zmínku stojí také hra Rogue, která si oproti výše zmíněnému Beneath Apple Manor získala výrazně větší popularitu a měla zásadní vliv na další vývoj herního designu. Právě podle této hry vznikl podžánr označovaný jako roguelike, který jsme si popsali výše, a z jejích principů vychází dodnes.

Hra byla představena v roce 1980 a jejími autory jsou Michael Toy, Glenn Wichman a Ken Arnold. Původně byla vyvíjena pro unixové systémy a zobrazovala herní svět pomocí ASCII znaků v textovém terminálu. Jednotlivé prvky prostředí byly reprezentovány znaky, například hráč symbolem @, stěny znakem #,

nepřátelé různými písmeny abecedy. Tento způsob zobrazení nebyl pouze důsledkem technických omezení tehdejšího hardware, ale zároveň umožňoval poměrně flexibilní práci s herním světem a jeho generováním.

Rogue bývá popisována jako jedna z prvních roguelike her, která výrazně zpopularizovala princip permanentní smrti (permadeath). Postup hrou nebylo možné ukládat a jakmile hráč zemřel, musel začít zcela od začátku. Hráči tak zůstávaly pouze nabyté zkušenosti a znalost herních mechanik. Smrt zde proto nepředstavovala pouze neúspěch, ale také nedílnou součást učení se hernímu systému.

Cílem hry je najít Amulet of Yendor, který se nachází v nejhlubším patře dungeonů, a následně se vrátit zpět na povrch. Dungeon je tvořen několika patry (tradičně 26), přičemž každé z nich je generováno procedurálně. Patra se skládají z místností propojených chodbami a jejich rozložení se při každém novém průchodu mění. Náhodně jsou rozmístěni také nepřátelé, pasti, poklady, zbraně a další předměty.

Prvek náhody se projevuje i ve vlastnostech předmětů. Například lektvary, svitky či prsteny mají při každé nové hře odlišné označení a jejich účinek není hráči předem znám. I pokud hráč nalezne stejný typ předmětu jako v předchozí hře, jeho vlastnosti se mohou lišit. Tím je posílena nejistota a nutnost experimentování.

Kombinace procedurálně generovaných úrovní a permanentní smrti zajišťuje, že každý průchod hrou je unikátní. Hráč se tak nemůže spoléhat na zapamatování konkrétní mapy, ale musí se přizpůsobovat aktuálně vygenerovaným podmínkám. Právě tento princip se stal jedním ze základních stavebních kamenů celého roguelike žánru [4, 5].

2.2.3 Elite

Dalším významným titulem využívajícím procedurální generování je hra Elite z roku 1984, kterou vytvořili David Braben a Ian Bell. Na rozdíl od předchozích her se nejedná o dungeon, ale o vesmírný simulátor obchodování a soubojů. Přesto je Elite z hlediska procedurálního generování velmi důležitým milníkem.

Hra byla původně vydána pro počítač BBC Micro, který disponoval velmi omezenou pamětí. Z tohoto důvodu nebylo možné ukládat rozsáhlý herní svět přímo v paměti. Vývojáři proto zvolili přístup, kdy byl herní svět generován algoritmicky na základě předem definovaného číselného základu (seed). Pomocí tohoto deterministického postupu bylo možné při každém spuštění znovu vytvořit stejnou galaxii bez nutnosti jejího explicitního uložení.

Elite obsahovalo osm galaxií, přičemž každá z nich zahrnovala stovky hvězdných systémů. Vlastnosti jednotlivých systémů, jako je jejich název, ekonomika, úroveň technologického rozvoje či typ vlády, byly generovány pomocí pseudonáhodných algoritmů. Procedurálně byly generovány také ceny komodit a další herní parametry.

Tento přístup ukázal, že procedurální generování není omezeno pouze na tvorbu dungeonů nebo jednotlivých úrovní, ale může být využito i pro vytvo-

ření rozsáhlých herních světů. Elite tak demonstrovalo, že algoritmická generace obsahu představuje efektivní řešení zejména v podmínkách omezených výpočetních a paměťových zdrojů [6].

2.3 Generování dungeonů ve 2D hrách

Tato kapitola se zaměřuje konkrétně na generování dungeonových map ve 2D hrách. Dungeonové mapy se typicky skládají z místností a chodeb, obsahují jasně definované průchozí a neprůchozí oblasti a musí splňovat požadavky na hratelnost. Generování takových struktur proto vyžaduje algoritmy, které dokáží zajistit nejen variabilitu výsledku, ale i splnění základních strukturálních vlastností.

Nejprve budou definovány požadavky kladené na generování dungeonových map. Následně budou představeny vybrané metody, které tvoří teoretický základ pro implementaci popsanou v dalších kapitolách.

2.3.1 Požadavky na generování dungeonových map

Při návrhu algoritmu pro generování dungeonové mapy je nutné zohlednit několik základních požadavků, které vycházejí jak z technických omezení, tak z hlediska hratelnosti. Generovaná mapa by neměla být pouze náhodným shlukem místností a chodeb, ale strukturovaným prostředím, které umožňuje smysluplný průchod a interakci.

Jedním ze základních požadavků je průchodnost mapy neboli connectivity. Všechny klíčové části mapy by měly být vzájemně dosažitelné, aby nedocházelo k izolovaným oblastem bez možnosti přístupu. Tento požadavek je zásadní zejména u her, kde je cílem projít celou úroveň nebo nalézt konkrétní objekt.

Dalším důležitým aspektem je kontrola struktury a rozložení prostoru. Mapa by měla obsahovat přiměřený poměr otevřených a uzavřených prostor, vhodnou velikost místností a dostatečně široké průchody, aby hráč mohl bez problémů projít. Příliš husté nebo naopak příliš prázdné prostředí může negativně ovlivnit hratelnost i orientaci hráče.

Významným požadavkem je rovněž variabilita výsledků. Opakované spuštění generátoru by mělo vést k odlišným mapám, aby byl zachován prvek znovuhratelnosti. Zároveň je však vhodné, aby generování respektovalo určitá pravidla nebo omezení, která zajistí konzistentní kvalitu výsledku.

Z technického hlediska je důležitá také výpočetní náročnost algoritmu. Generování by mělo probíhat v přiměřeném čase a nemělo by výrazně zatěžovat systém, zejména pokud je realizováno za běhu hry (tzv. online generování). U jednodušších 2D dungeonových her je obvykle možné využít algoritmy s relativně nízkou časovou složitostí.

V neposlední řadě je vhodné umožnit parametrizaci generátoru. Možnost nastavit velikost mapy, počet místností, hustotu chodeb nebo další vlastnosti umožňuje lépe přizpůsobit výsledné prostředí konkrétním požadavkům hry a zároveň usnadňuje testování různých konfigurací [1, 7].

2.4 Přehled metod generování

Na základě výše uvedených požadavků a pravidel lze identifikovat několik přístupů a algoritmů generování dungeonových map. Tyto metody se liší zejména způsobem konstrukce prostoru, mírou kontroly, výslednou strukturou a výpočetní náročností. Některé algoritmy vytvářejí mapu postupně přidáváním jednotlivých prvků, jiné pracují s rozdělováním prostoru nebo s pravidly aplikovanými na mřížku buněk.

Ve 2D hrách se nejčastěji setkáváme s metodami založenými na náhodném umísťování místností a jejich následném propojování, s rozdělováním velkého prostoru na menší části nebo s algoritmy založenými na takzvané náhodné procházce. Každý z těchto přístupů má své výhody i omezení a je vhodný pro odlišný typ herního prostředí. Porovnáním efektivity vybraných kombinací těchto algoritmů z hlediska času generování a spotřeby výpočetních zdrojů se zabývá například práce [8].

Cílem této podkapitoly je stručně představit princip fungování vybraných metod, jejich základní charakteristiky a typické oblasti využití. Detailnější rozpracování a samotná implementace vybraných algoritmů budou popsány v následujících kapitolách.

2.4.1 Náhodné umístění místností a chodeb

Generování pomocí náhodného umístění místností a chodeb patří mezi nejpoužívanější a zároveň nejjednodušší přístupy z hlediska implementace.

Základní princip spočívá v tom, že se v rámci mapy nejprve generují místnosti různých velikostí, obvykle obdélníkového nebo čtvercového tvaru. Během generování se kontroluje, aby se nově vznikající místnost nepřekrývala s již existujícími. Výsledkem je množina místností reprezentovaných například souřadnicemi a rozměry. Následně jsou jednotlivé místnosti propojeny pomocí chodeb. Ty bývají realizovány jako jednoduché horizontální a vertikální průchody spojující středy dvou místností nebo jejich nejbližší body. Častým řešením je také propojení ve tvaru písmene L, kdy jsou dvě místnosti spojeny dvojicí kolmých chodeb.

Pro zajištění průchodnosti mapy a dosažitelnosti všech místností se často využívají algoritmy z oblasti teorie grafů. Jedním z typických řešení je použití minimální kostry grafu (Minimum Spanning Tree), která zaručuje propojení všech místností při minimálním počtu chodeb.

Proces generování lze obecně rozdělit do několika kroků:

1. generování náhodných místností v rámci mapy,
2. kontrola jejich překrývání,
3. výběr místností určených k propojení,
4. generování chodeb mezi jednotlivými místnostmi.

Výhodou této metody je především její jednoduchost. Generované dungeony obvykle obsahují přehledně oddělené místnosti a rozumitelnou síť propojení. Nevýhodou může být menší variabilita, protože místnosti mají často pravidelný tvar obdélníků a čtverců, a prostředí tak může působit příliš uniformně. Dalším problémem bývá méně efektivní využití prostoru, kdy mezi jednotlivými místnostmi zůstávají větší nevyužité oblasti. Komplikace může přinášet i samotné propojování. U jednodušších implementací může docházet ke kolizím chodeb, nevhodnému křížení nebo ke vzniku příliš dlouhých průchodů, které narušují dynamiku pohybu v prostředí.

Z těchto důvodů bývá metoda často kombinována s dalšími postupy nebo doplněna o dodatečná pravidla, která omezují velikosti a vzájemné vzdálenosti místností, délku chodeb nebo celkové rozložení prostoru [1, 7].

2.4.2 Binary Space Partitioning

Metoda Binary Space Partitioning (BSP) patří mezi přístupy založené na systematickém rozdělování prostoru. Základní princip spočívá v tom, že se celá mapa nejprve reprezentuje jako jeden velký obdélníkový nebo čtvercový prostor, který je následně postupně rozdělován na menší části. Každé rozdělení může probíhat horizontálně nebo vertikálně a výsledkem je hierarchická struktura podobná binárnímu stromu.

Samotný proces generování obvykle začíná rozdělením celé mapy na dvě části. Tyto části se dále rekurzivně dělí, dokud nedosáhnou určité minimální velikosti. Každé rozdělení vytváří dva nové uzly ve stromové struktuře, přičemž listové uzly představují konečné oblasti, ve kterých mohou být vytvořeny místnosti. Takto vzniklá stromová reprezentace umožňuje přehledně uchovávat strukturu rozdělení prostoru.

V jednotlivých vzniklých oblastech jsou následně vytvořeny místnosti, jejichž velikost je o něco menší než velikost dané oblasti. Tím vzniká přirozený prostor mezi místnostmi, který může být využit například pro generování chodeb nebo dalších herních prvků. Po vytvoření místností je nutné zajistit jejich propojení. To se obvykle provádí postupným propojováním místností mezi sousedními částmi stromové struktury, často od nejnižších úrovní stromu směrem nahoru.

Díky tomuto postupu lze relativně snadno zajistit vzájemnou dosažitelnost všech místností. Zásadní výhodou BSP je poměrně dobrá kontrola nad velikostí a rozmístěním jednotlivých částí mapy. Parametry dělení prostoru umožňují ovlivnit výsledný vzhled prostředí a předejít situacím, kdy by vznikaly příliš malé nebo naopak příliš velké místnosti.

Nevýhodou této metody může být opět poměrně pravidelný a místy uniformní vzhled generovaného prostředí, protože výsledná struktura často odráží samotný způsob dělení prostoru. Mapy vytvořené pomocí BSP tak mohou působit méně přirozeně než mapy generované například pomocí buněčných automatů [1, 7].

2.4.3 Metoda náhodné procházky

Metoda náhodné procházky, často označovaná také jako *Drunkard's Walk*, je založena na jednoduchém principu postupného rozšiřování prostoru pomocí náhodného pohybu. Algoritmus začíná na určité počáteční pozici v mapě a v každém kroku se náhodně přesune do jednoho ze sousedních polí. Směr pohybu je obvykle vybírán ze čtyř základních směrů v mřížce, tedy nahoru, dolů, doleva nebo doprava.

Při každém kroku je aktuální buňka označena jako průchozí prostor. Postupným opakováním tohoto procesu vzniká síť chodeb a otevřených prostor, jejichž tvar závisí na délce a průběhu náhodné procházky. Výsledná struktura mapy bývá velmi nepravidelná a rozdílná při každém generování.

Proces generování obvykle pokračuje po předem stanovený počet kroků nebo dokud není dosaženo určitého poměru průchozích buněk v mapě. V některých variantách algoritmu může být použito více nezávislých „agentů“, kteří se po mapě pohybují současně a vytvářejí tak komplexnější strukturu prostředí.

Výhodou této metody je velmi jednoduchá implementace a schopnost generovat přirozeně působící struktury. Nevýhodou je naopak menší kontrola nad výsledným tvarem mapy a obtížnější zajištění rovnoměrného využití prostoru. V některých případech může docházet k tomu, že většina průchozích oblastí vzniká v blízkosti počáteční pozice algoritmu, zatímco jiné části mapy zůstávají téměř nevyužité.

Podobně jako u buněčných automatů se tato metoda často používá při generování jeskynních nebo jinak přirozeně působících prostředí [1].

2.4.4 Buněčné automaty

Buněčné automaty představují přístup k procedurálnímu generování založený na aplikaci lokálních pravidel na mřížku buněk. Mapa je v tomto případě reprezentována jako dvourozměrná mřížka, kde každá buňka může být například ve stavu stěna nebo průchozí prostor. Na začátku generování je mapa obvykle inicializována náhodným rozdělením těchto stavů, což vytváří základní strukturu prostředí.

V následujících iteracích se stav jednotlivých buněk aktualizuje na základě stavu jejich sousedních buněk. Nejčastěji se používá tzv. Mooreovo sousedství, které zahrnuje osm buněk obklopujících aktuální buňku. Pravidla generování mohou například určovat, že buňka zůstane průchozí pouze tehdy, pokud má dostatečný počet průchozích sousedů, případně že se změní na stěnu, pokud je obklopena větším množstvím stěn.

Opakovaným aplikováním těchto pravidel dochází k postupnému vyhlazování náhodně vytvořené struktury. Izolované buňky nebo malé skupiny buněk se postupně odstraňují a vznikají větší souvislé oblasti. Výsledkem jsou mapy s nepravidelnými tvary, které často připomínají přirozené jeskynní systémy.

Výhodou této metody je schopnost generovat organicky působící prostředí s nepravidelnými tvary, které se liší od klasických dungeonů založených na místnos-

tech a chodbách. Nevýhodou může být menší kontrola nad přesnou strukturou mapy a také nutnost dodatečných kroků, které zajistí průchodnost celé mapy. V některých případech je proto nutné odstranit izolované oblasti nebo dodatečně propojit jednotlivé části prostředí [1, 7].

2.4.5 Metoda bludiště

Metody generování bludišť vytvářejí struktury tvořené převážně úzkými chodbami, které tvoří souvislou síť průchodů. Mapa je v tomto případě obvykle reprezentována jako mřížka buněk oddělených stěnami. Samotné generování spočívá v postupném odstraňování těchto stěn tak, aby vznikla síť propojených průchodů.

Tyto algoritmy často vycházejí z principů grafových algoritmů, například prohledávání do hloubky (Depth-First Search) s návratem (backtracking) nebo algoritmů typu Prim či Kruskal. V případě algoritmu založeného na prohledávání do hloubky se začíná v náhodně zvolené buňce, ze které se postupně prochází do sousedních dosud nenavštívených buněk. Při každém kroku se odstraní stěna mezi aktuální buňkou a vybraným sousedem. Pokud již neexistuje žádný nenavštívený soused, algoritmus se vrací zpět k předchozí buňce.

Primův algoritmus vytváří bludiště postupným rozšiřováním již vytvořené oblasti. Algoritmus začíná v náhodné buňce a postupně vybírá sousední stěny, které oddělují navštívené buňky od nenavštívených. Pokud odstranění vybrané stěny spojí dvě dosud nepropojené oblasti, je tato stěna odstraněna a nová buňka se stane součástí bludiště.

Kruskalův algoritmus naproti tomu pracuje s množinou všech stěn v mapě. Stěny jsou postupně vybírány v náhodném pořadí a odstraňují se pouze tehdy, pokud jejich odstranění nespojí dvě buňky, které již patří do stejné části bludiště. Tím se postupně vytváří souvislá síť průchodů bez vzniku cyklů.

Výsledkem je tzv. perfektní bludiště, ve kterém mezi dvěma body existuje právě jedna cesta. Taková struktura neobsahuje cykly a vytváří spleť chodeb. Tento typ generování je vhodný zejména pro hry zaměřené na průzkum a orientaci v prostoru.

Nevýhodou této metody je omezený výskyt větších otevřených prostor, protože generovaná struktura je tvořena převážně úzkými chodbami. Z tohoto důvodu bývá v dungeonových hrách často kombinována s jinými metodami, které umožňují vytvářet také větší místnosti. V některých případech se proto po vygenerování bludiště odstraňují vybrané stěny, čímž vznikají cykly a otevřenější struktura mapy [1, 7].

2.4.6 Další metody

Kromě výše uvedených přístupů existuje celá řada dalších metod generování dungeonových map. Patří mezi ně například generování založené na grafech, kde je struktura mapy nejprve reprezentována jako graf místností a jejich propojení. Jednotlivé vrcholy grafu představují místnosti a hrany určují jejich vzájemné

propojení. Na základě takto vytvořené abstraktní struktury je následně vytvořena konkrétní geometrická podoba mapy.

Další možností je využití procedurálních gramatik, které definují pravidla pro vytváření jednotlivých částí mapy. Generování poté probíhá aplikací těchto pravidel, podobně jako při tvorbě vět v formálních jazycích. Tento přístup umožňuje poměrně přesně řídit strukturu generovaného prostředí a je vhodný zejména pro hry, které vyžadují specifické uspořádání jednotlivých herních prvků.

V literatuře se rovněž objevují přístupy založené na evolučních algoritmech nebo optimalizačních metodách, které se snaží generované mapy postupně vylepšovat podle předem definovaných kritérií. Tyto metody pracují s populací kandidátních map, které jsou postupně upravovány pomocí genetických operací, jako je mutace nebo křížení. Výsledné mapy jsou následně hodnoceny podle zvolených metrik, například průchodnosti, velikosti prostoru nebo rozložení herních prvků.

Modernější přístupy zahrnují také metody založené na splňování různých omezení. Jedná se o takzvanou *constraint-based generation* nebo algoritmy typu *Wave Function Collapse*, které generují mapy postupným výběrem kompatibilních prvků z předem definované množiny vzorů. Tyto metody umožňují generovat velmi rozmanité struktury, avšak jejich implementace bývá složitější.

Uvedené přístupy jsou často výpočetně náročnější než základní algoritmy popsané v předchozích podkapitolách, a proto se v praxi využívají spíše při offline generování obsahu nebo při návrhu komplexnějších herních světů [1, 7].

2.5 Shrnutí způsobů generování

V této kapitole byly představeny vybrané metody procedurálního generování dungeonových map. Jednotlivé přístupy se liší především způsobem konstrukce prostoru, mírou kontroly nad výslednou strukturou a typem generovaného prostředí. Zatímco některé metody vytvářejí spíše pravidelné struktury místností a chodeb, jiné generují nepravidelná a organicky působící prostředí.

Základní charakteristiky popsaných metod shrnuje tabulka 1. Podrobnější porovnání implementovaných algoritmů bude provedeno v kapitole věnované jejich vyhodnocení.

Metoda	Kontrola struktury	Variabilita	Typ prostředí
Místnosti a chodby	vysoká	střední	klasické dungeony
Binary Space Partitioning	vysoká	střední	strukturované mapy
Buněčné automaty	nízká	vysoká	jeskyně
Náhodná procházka	nízká	vysoká	organické struktury
Bludiště	střední	střední	labyrinty

Tabulka 1: Srovnání vybraných metod generování dungeonových map

Na základě provedené rešerše byly pro praktickou část práce vybrány algoritmy generování pomocí místností a chodeb, Binary Space Partitioning, buněčné

automaty, metoda náhodné procházky a metoda generování bludiště. Tyto algoritmy reprezentují různé přístupy k procedurálnímu generování map a díky tomu je lze snadno vzájemně porovnat.

3 Experimentální část

Tato kapitola se věnuje návrhu systému pro generování dungeonových map a technologiím použitým při implementaci vytvořeného prototypu. Navržený systém slouží k demonstraci a porovnání vybraných metod procedurálního generování popsanych v předchozí kapitole.

Nejprve jsou představeny nástroje využívané při vývoji aplikace. Následně je popsán samotný návrh systému, zejména způsob reprezentace dungeonové mapy, struktura generátoru a způsob vizualizace výsledného prostředí.

3.1 Použité technologie

V této kapitole budou představeny využívané technologie pro implementaci výše zmíněných metod.

3.1.1 Herní engine Godot

Pro vizualizaci generátoru dungeonových map byl zvolen herní engine Godot [9]. Jedná se o open-source engine určený pro vývoj 2D i 3D her. Poskytuje integrované nástroje pro práci s grafikou, fyzikou, scénami i skriptováním, což umožňuje rychlé vytváření a testování herních prototypů.

Významnou vlastností Godotu je podpora dlaždicových map (TileMap), které jsou při vývoji 2D her často využívány a výrazně to usnadňuje samotné grafické ztvárnění. Tento způsob reprezentace prostředí je zároveň vhodný i pro implementaci algoritmů procedurálního generování, protože umožňuje jednoduchou manipulaci s jednotlivými buňkami mapy.

Engine zároveň umožňuje snadné spouštění a testování různých algoritmů v jednom prostředí, což je výhodné zejména při jejich vzájemném porovnávání.

Pro implementaci algoritmů byl použit integrovaný skriptovací jazyk GDScript [10], který je syntakticky podobný jazyku Python. GDScript umožňuje přímý přístup k objektům herní scény, komponentám enginu i datovým strukturám, což usnadňuje rychlé prototypování generovacích metod a jejich průběžné ladění přímo v prostředí enginu. Při vývoji podpůrných částí aplikace (ovládání kamery, herní mechaniky) byl jako pomocný nástroj využit GitHub Copilot; samotné generovací algoritmy jsou vlastní implementací na základě publikovaných algoritmických popisů.

3.2 Návrh systému generování

Při návrhu samotného systému generování bylo nutné předem stanovit, jaké metody chceme zahrnout, jak bude vypadat mapa, například velikost a případně počet dlaždic a jak bude ztvárněno ovládání samotné hry. V dalších kapitolách bude implementace rozebrána detailněji.

3.2.1 Reprezentace mapy

Všechny implementované metody generování pracují nad společnou datovou reprezentací mapy ve formě dvourozměrného pole. Mapa je uložena jako pole řádků, kde každý prvek odpovídá jedné buňce dungeonové mapy. Jednotlivé buňky obsahují celočíselnou hodnotu určující typ dlaždice.

V implementaci jsou rozlišeny tři základní stavy buněk: prázdný prostor, podlaha a stěna. Tyto stavy jsou reprezentovány číselnými hodnotami 0, 1 a 2, které odpovídají prázdné buňce, průchozí podlaze a stěně. Právě tyto hodnoty následně slouží jako základ pro vykreslení odpovídajících dlaždic.

Hodnota 0 se během generování používá především jako výchozí stav mapy, zatímco hodnoty 1 a 2 představují výslednou strukturu dungeonového prostředí.

Příklad jednoduché reprezentace mapy je uveden v tabulce 2. Ukázka ilustruje průchozí oblast (1), která je obklopena stěnami (2), zatímco zbytek mapy tvoří nevyužitý prostor reprezentovaný hodnotou 0.

0	0	0	0	0
0	2	2	2	0
2	2	1	2	2
2	1	1	1	2
2	2	2	2	2

Tabulka 2: Ukázka reprezentace mapy v mřížce.

Velikost mapy je určena dvěma parametry, šířkou a výškou, které jsou předávány generovacím algoritmům při jejich spuštění. Jednotlivé metody následně vytvářejí a upravují strukturu mapy podle svého principu generování.

Zvolený způsob reprezentace umožňuje sjednotit práci všech implementovaných generovacích algoritmů, protože každá metoda vrací výslednou mapu ve stejném formátu bez ohledu na svůj vnitřní princip. Výsledná mapa tak může být následně jednotným způsobem zpracována při vizualizaci i při vyhodnocení výsledků.

Výhodou zvoleného řešení je především jeho jednoduchost a přehlednost. Přístup k jednotlivým buňkám je realizován pomocí jejich souřadnic, což usnadňuje jak samotné generování mapy, tak i další operace při práci s mapou.

3.2.2 Struktura generátoru

Systém byl navržen modulárně tak, aby bylo možné jednotlivé algoritmy snadno rozšiřovat a vzájemně porovnávat. Každá metoda je tedy implementována v sa-

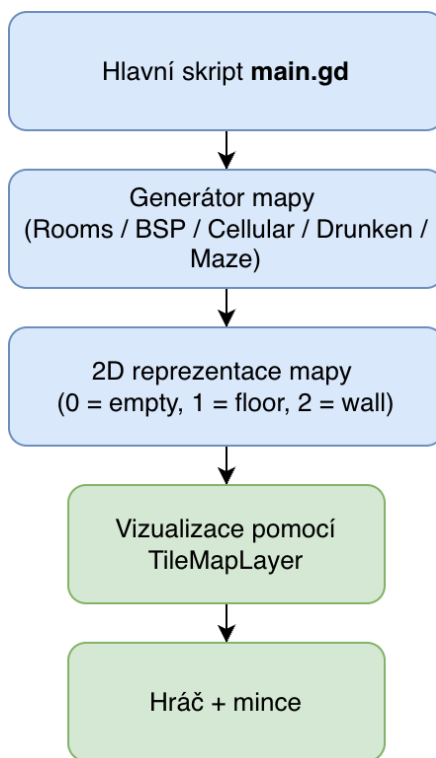
mostatném skriptu a využívá jednotné rozhraní v podobě funkce `generate(width, height)`, která pro zadané rozměry vrací datovou reprezentaci mapy.

Díky tomuto sjednocení lze do aplikace přidávat další generátory bez zásahů do ostatních částí systému. V rámci vytvořeného prototypu byly takto implementovány metody založené na náhodném umístění místností, Binary Space Partitioning, buněčných automatech, náhodné procházce a generování bludiště.

Na úrovni aplikace jejich běh řídí skript `main.gd`, který podle uživatelské volby vybere odpovídající algoritmus, spustí jeho běh a převezme výslednou mapu k dalšímu zpracování. Následně je tato reprezentace předána vrstvě zajišťující vizualizaci a grafické znázornění zvoleného generování je uživateli vykresleno.

Navržená architektura tím odděluje samotnou logiku tvorby mapy od jejího vykreslení. Generátory pracují pouze s daty, zatímco zobrazení výsledku obstarávají samostatné komponenty. Samotné vykreslování tedy neovlivňuje dobu trvání algoritmu, což je žádoucí pro následné porovnávání.

Architektura navrženého systému je schematicky znázorněna na obrázku 1.



Obrázek 1: Schéma architektury systému generování dungeonových map

3.2.3 Vizualizace mapy a hráče

Pro vizualizaci generovaných map byly v aplikaci použity grafické prvky, takzvané *asset*y. Tyto *asset*y slouží především k přehlednému zobrazení struktury generované mapy a usnadňují orientaci uživatele v herním prostředí. Vizualizace zahrnuje grafiku podlahy, stěn, postavy hráče a sbíratelných objektů.

Zobrazení mapy je realizováno pomocí komponenty `TileMapLayer`. Po dokončení generování je datová reprezentace mapy postupně převedena na odpovídající dlaždice, přičemž jednotlivé typy buněk jsou mapovány na konkrétní grafické prvky představující podlahu nebo stěnu. V zobrazovacím prostředí bylo zvoleno postupné vykreslování mapy po jednotlivých řádcích. Tento způsob sice není nezbytný z hlediska samotného generování, ale usnadňuje vizuální kontrolu výsledků a umožňuje lépe sledovat podobu jednotlivých algoritmů.

Součástí prototypu je také jednoduchá reprezentace hráče, která slouží především k ověření průchodnosti mapy. Hráč je po vygenerování umístěn na vhodnou průchozí pozici tak, aby měl kolem sebe dostatek prostoru a mohl se pohybovat. Je tedy upřednostněna oblast, která poskytuje dostatek volného prostoru v okolí, aby nedocházelo ke kolizi se stěnami ihned po vytvoření mapy.

Pro doplnění testovacího prostředí byly do mapy přidány také sbíratelné objekty ve formě mincí, které jsou po vygenerování náhodně rozmístěny na průchozích dlaždicích. Tyto prvky neslouží jako součást generování samotného, ale usnadňují praktické ověření funkčnosti a pohybu v prostředí a zároveň působí uživatelsky více přívětivě.

V rámci implementace byl využit volně dostupný balíček *2D Pixel Dungeon Asset Pack* [11], jehož autorem je grafik vystupující pod jménem `Pixel_Poem`. Tento balíček obsahuje základní sadu grafických prvků určených pro tvorbu dungeonových roguelike her.

Použité assety mají podobu pixelové grafiky v top-down perspektivě, tedy seshora dolů, a jsou navrženy právě pro práci s dlaždicovými mapami (tilemap), což umožňuje jejich snadnou integraci do herního enginu. Balíček je poskytován jako volně dostupný asset pack, který lze využít v nekomerčních i komerčních projektech.

Vzhledem k tomu, že hlavním cílem práce je implementace a porovnání algoritmů procedurálního generování map, slouží grafická vrstva pouze jako podpůrný nástroj pro vizualizaci a prezentaci výsledků generování.

3.2.4 Ovládání generování a zobrazení statistik

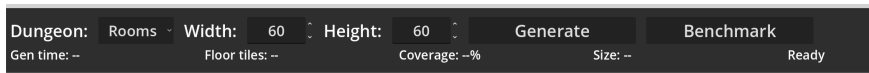
Součástí aplikace je jednoduché uživatelské rozhraní, které umožňuje výběr konkrétní generovací metody a nastavení základních parametrů mapy, zejména její šířky a výšky.

Uživatelské rozhraní obsahuje dvě hlavní ovládací tlačítka, *Generate* a *Benchmark*. Tlačítko *Generate* slouží k vizuálnímu spuštění generování mapy pomocí vybraného algoritmu. Po dokončení generování jsou uživateli zobrazeny základní statistické údaje, které charakterizují vlastnosti vygenerované mapy. Mezi tyto údaje patří zejména čas potřebný pro vytvoření mapy, počet průchozích dlaždic a procentuální pokrytí mapy průchozím prostorem. Generování mapy je zároveň vizualizováno postupným vykreslováním jednotlivých řádků, což umožňuje sledovat průběh tvorby výsledného prostředí.

Tlačítko *Benchmark* slouží k automatizovanému testování implementovaných generovacích metod. Po jeho spuštění aplikace postupně spustí všechny imple-

mentované algoritmy pro předem definované velikosti map. V rámci každé konfigurace je generování provedeno opakovaně, aby bylo možné získat reprezentativní hodnoty měřených parametrů. Na rozdíl od běžného generování není při tomto režimu mapa vizualizována, což umožňuje přesnější měření času generování.

Výsledky jednotlivých běhů jsou ukládány do souboru `results.csv`, jehož struktura je podrobněji popsána v podkapitole věnované metodice měření.



Obrázek 2: Uživatelské rozhraní aplikace pro generování map

3.3 Implementace generovacích metod

Principy jednotlivých algoritmů generování map byly popsány v kapitole 2.4. V následujících podkapitolách je popsána jejich konkrétní implementace v rámci vytvořené aplikace. Jednotlivé skripty se nacházejí v adresáři `res://dungeons/algorithms`. Kompletní zdrojové kódy jednotlivých generátorů jsou uvedeny v příloze A.

3.3.1 Náhodné generování místností a chodeb

Implementaci algoritmu najdeme v souboru `rooms_generator.gd` (viz zdrojový kód 1). Na začátku funkce `generate(width, height)` je vytvořen lokální generátor náhodných čísel `RandomNumberGenerator`. Použití lokální instance zajišťuje, že generování mapy neovlivňuje jiné části aplikace využívající náhodnost. Následně je inicializována prázdná mapa o zadaných rozměrech, přičemž všechny buňky jsou nastaveny na hodnotu `TILE_EMPTY`. Tím vznikne výchozí reprezentace mapy, do níž budou postupně zapisovány místnosti, chodby a stěny.

Počet generovaných místností je odvozen od celkové plochy mapy podle vzorce `room_count = max(5, width * height / 360)`. Na malých mapách je tak vygenerováno alespoň pět místností, zatímco na velkých mapách počet místností úměrně roste s plochou. Maximální počet pokusů o umístění místnosti je stanoven jako `room_count * 10`. Každá nová místnost je reprezentována obdélníkem typu `Rect2`. Její šířka a výška jsou generovány náhodně v intervalu od 8 do 16 dlaždic. Pozice levého horního rohu místnosti je následně určena tak, aby místnost zůstala uvnitř hranic mapy a zároveň nebyla umístěna přímo na jejím okraji.

Před přidáním nové místnosti do mapy je provedena kontrola překryvu se všemi dříve umístěnými místnostmi. Tato kontrola je realizována pomocí metody `grow(1).intersects(other)`, což znamená, že se testuje nejen skutečný průnik obdélníků, ale i jejich rozšíření o jednu dlaždici na každou stranu. V praxi tak mezi dvěma místnostmi vždy zůstává alespoň jednopolíčková mezera.

Pokud by se nová místnost překrývala s některou z již existujících, je zahozena a algoritmus pokračuje dalším pokusem.

V případě, že nová místnost překryv nevykazuje, je přidána do seznamu místností a její vnitřní plocha je zapsána do mapy jako podlaha. To je realizováno dvojicí vnořených cyklů, které procházejí všechny buňky uvnitř obdélníku místnosti a nastavují jejich hodnotu na `TILE_FLOOR`. Tím dojde k vyznačení průchozí oblasti odpovídající nové místnosti.

Každá nová místnost je následně propojená chodbou. První místnost je však z tohoto propojování vyloučena. Pro propojení se využívají středy obou místností získané pomocí metody `get_center()`. Vlastní vytvoření chodby zajišťuje pomocná funkce `_carve_corridor()`, která mezi dvěma zadanými body vyznačí chodbu tvaru písmene L. Algoritmus náhodně rozhoduje, zda bude chodba nejprve vedena ve vodorovném a poté ve svislém směru, nebo opačně. Díky tomu výsledné mapy nepůsobí zcela uniformně.

Chodba není tvořena pouze jednou dlaždicí, ale má nastavitelnou šířku. V implementaci je výchozí šířka parametru `corridor_width` nastavena na hodnotu 2. Funkce proto při vykreslování chodby neoznačuje pouze centrální linii, ale také sousední buňky v jejím okolí. Výsledná chodba je tak vizuálně širší a průchod mapou je přehlednější. Při zápisu dlaždic chodby je současně kontrolováno, zda se zapisované souřadnice nacházejí uvnitř hranic mapy, a to pomocí pomocné funkce `_in_bounds()`.

Po dokončení umístování místností a chodeb následuje doplnění stěn. Tuto operaci provádí funkce `_add_walls()`, která prochází všechny buňky mapy a u každé dlaždice podlahy kontroluje její osmibuněčné okolí. Pokud se v sousedství nachází buňka označená jako prázdný prostor, je její hodnota změněna na `TILE_WALL`. Tímto způsobem vznikne souvislé ohraničení všech průchozích částí mapy bez nutnosti explicitně generovat stěny již během vytváření místností a chodeb.

3.3.2 Binary Space Partitioning

Implementace metody BSP se nachází v souboru `bsp_generator.gd` (viz zdrojový kód 2). Skript využívá rekursivní rozdělování prostoru mapy na menší oblasti pomocí binární stromové struktury. Každý uzel stromu reprezentuje obdélníkovou oblast mapy, která může být dále rozdělena na dvě pod-oblasti.

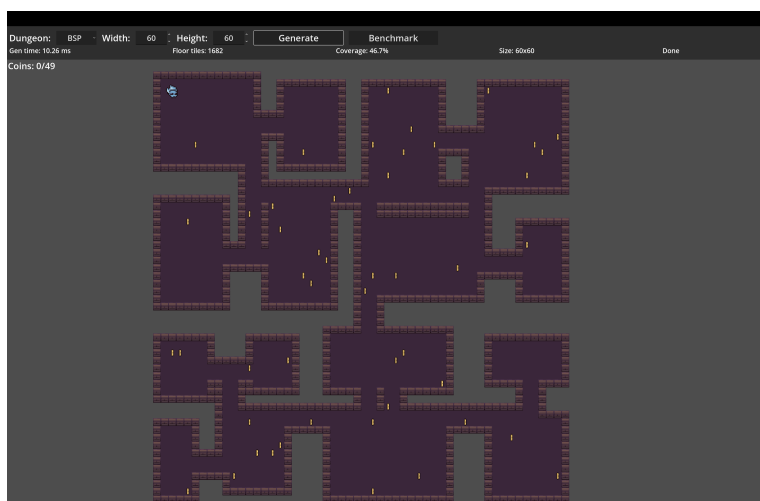
Generování začíná vytvořením počáteční oblasti odpovídající celé velikosti mapy. Tato oblast představuje kořen stromu a je postupně rozdělena na dvě části. Směr dělení je určován podle poměru šířky a výšky oblasti. Pokud je oblast výrazně širší než vyšší, je preferováno vertikální dělení, v opačném případě horizontální. Pokud jsou rozměry oblasti podobné, je směr dělení zvolen náhodně.

Při každém dělení je zvolena pozice řezu tak, aby vzniklé podoblasti splňovaly minimální požadovanou velikost. Tím je zajištěno, že v nich bude možné vytvořit místnost. Každé rozdělení vytváří dva nové uzly stromu, které jsou dále zpracovávány stejným způsobem.

Rekurzivní dělení pokračuje do okamžiku, kdy další rozdělení již není možné kvůli minimální velikosti oblasti. Takové uzly představují listy stromu. V každém listu je následně vytvořena místnost, jejíž velikost i pozice jsou generovány náhodně uvnitř příslušné oblasti.

Po vytvoření místností jsou jednotlivé části stromu postupně propojeny chodbami. Propojení probíhá při návratu z rekurzivního volání, kdy jsou vybrány vhodné body v místnostech patřících do dvou sousedních oblastí. Mezi těmito body je vytvořena chodba zajišťující průchod mezi jednotlivými částmi mapy.

Výsledná mapa je zapsána do dvourozměrného pole, které obsahuje informace o typu jednotlivých dlaždic. Průchozí části odpovídají podlahám místností a chodeb, zatímco okolní prostor je následně doplněn o stěny.



Obrázek 3: Ukázka mapy generované pomocí algoritmu BSP

3.3.3 Metoda náhodné procházky

Náhodná procházka je implementována v souboru `drunken_generator.gd` (viz zdrojový kód 3). Generování mapy probíhá opět ve funkci `generate(width, height)`, která vytváří dvourozměrné pole reprezentující výslednou mapu.

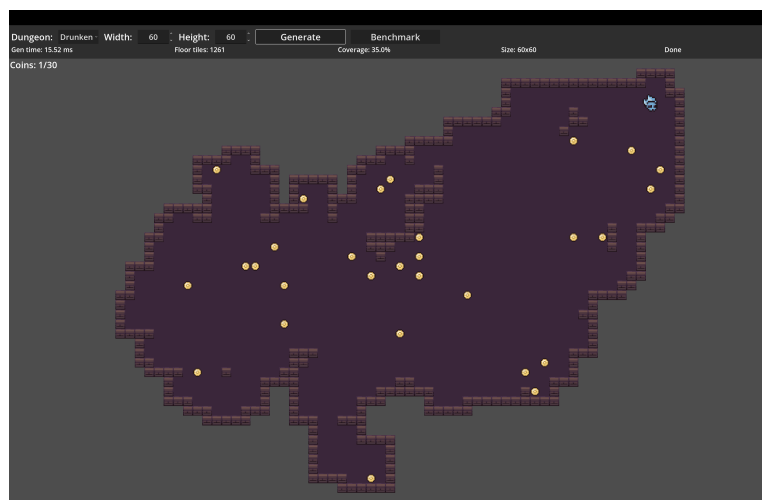
Na začátku funkce je vytvořena prázdná mapa o zadaných rozměrech, ve které jsou všechny buňky inicializovány jako neprůchozí prostor. Následně je vypočten cílový počet průchozích dlaždic, který je určen jako určitý podíl z celkové plochy mapy. Tento parametr určuje, jak velkou část mapy bude algoritmus postupně odkrývat.

Samotné generování probíhá pomocí simulace náhodného pohybu po mapě. Algoritmus začíná na náhodně zvolené pozici uvnitř mapy, která je označena jako průchozí dlaždice. V každém kroku je následně náhodně zvolen směr pohybu, typicky nahoru, dolů, doleva nebo doprava. Pokud se nová pozice nachází uvnitř hranic mapy, je tato buňka označena jako podlaha a aktuální pozice generátoru se přesune na tuto dlaždici.

Tento proces se opakuje, dokud počet vytvořených průchozích dlaždic nedosáhne předem definovaného cílového počtu. V důsledku náhodného pohybu vzniká nepravidelná struktura připomínající přirozeně vzniklé jeskynní prostory. Na rozdíl od metod založených na explicitním generování místností zde nevznikají jasně definované obdélníkové prostory, ale spíše organicky tvarované oblasti.

Po dokončení samotné náhodné procházky jsou kolem průchozích dlaždic doplněny stěny. Tento krok zajišťuje, že prázdné buňky sousedící s podlahou jsou označeny jako stěny, čímž vznikne souvislé ohraničení průchozí části mapy.

Vzhledem k tomu, že cílový počet průchozích dlaždic je určen předem, je výsledné pokrytí mapy pro danou velikost poměrně stabilní. Variabilita generovaných map se proto projevuje především ve tvaru a rozmístění vytvořených prostorů.



Obrázek 4: Ukázka mapy generované pomocí algoritmu náhodné procházky

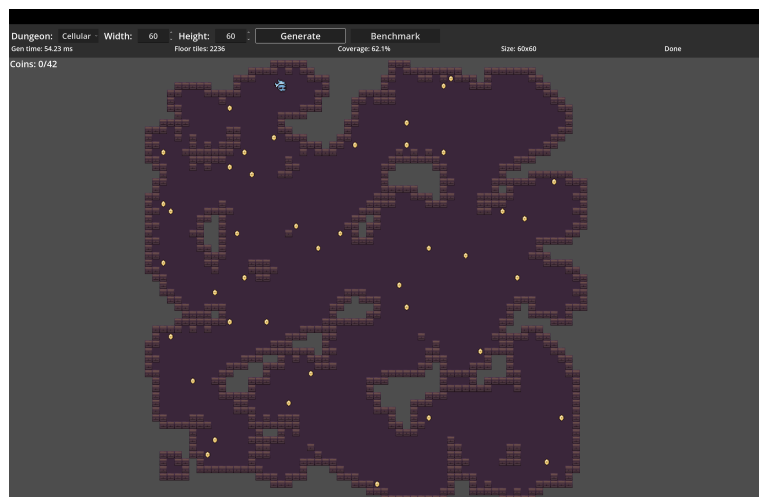
3.3.4 Buněčné automaty

Implementace buněčných automatů je v souboru `cellular_generator.gd` (viz zdrojový kód 4). První fáze vytvoří počáteční mapu náhodným vyplněním buněk podlahou podle pravděpodobnosti `FILL_PROBABILITY`. Okraje mapy zůstávají prázdné, aby se předešlo problémům při následném vyhodnocování sousedství.

V další fázi proběhne několik simulačních kroků. V každém kroku se vytváří nová mapa, aby se průběžné změny nepromítaly do výpočtu sousedů ve stejném kroku. Pro každou buňku se spočítá počet podlahových sousedů v Mooreově okolí a podle prahové hodnoty se rozhodne o novém stavu buňky. Opakováním těchto kroků se náhodný šum postupně vyhladí do organičtějších tvarů.

Po dokončení simulace je nad výslednou podlahou doplněna vrstva stěn. Výstupem jsou nepravidelné mapy jeskynního charakteru, které jsou vhodné pro porovnání s metodami generujícími pravidelnější struktury.

U map generovaných metodou buněčných automatů se mohou vyskytovat izolované průchozí oblasti. Z tohoto důvodu bylo při umísťování hráče nutné zohlednit souvislost průchozí plochy a upřednostnit umístění do největší spojitě oblasti mapy.



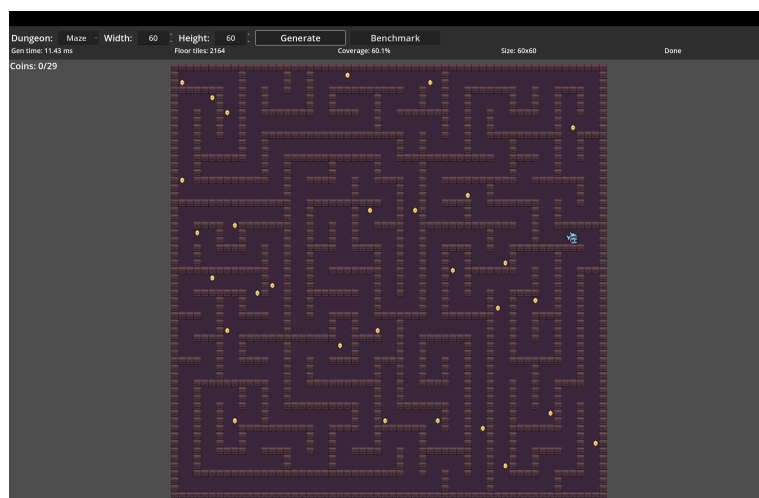
Obrázek 5: Ukázka mapy generované pomocí algoritmu buněčných automatů

3.3.5 Metoda bludiště

Metoda bludiště je implementována v souboru `maze_generator.gd` (viz zdrojový kód 5) a vychází z algoritmu *recursive backtracker* (DFS se zásobníkem). Vnitřní reprezentace není tvořena přímo dlaždicemi mapy, ale mřížkou abstraktních buněk, z nichž každá obsahuje bitovou masku čtyř stěn.

Generování začíná v počáteční buňce a postupně odstraňuje stěny mezi aktuální buňkou a náhodně vybraným nenavštíveným sousedem. Pokud aktuální buňka nemá dostupného nenavštíveného souseda, algoritmus se vrací zpět po zásobníku. Tento postup pokračuje, dokud nejsou navštíveny všechny buňky.

Po dokončení průchodu je buněčná reprezentace převedena do tile mapy. Každá buňka je vykreslena jako menší průchozí oblast a podle odstraněných stěn se mezi sousedními buňkami vytvářejí koridory. V závěru jsou stejně jako u ostatních metod doplněny stěny kolem podlahy.



Obrázek 6: Ukázka mapy generované pomocí algoritmu bludiště

3.4 Metodika měření a vyhodnocení

Pro objektivní porovnání implementovaných algoritmů byl navržen automatizovaný benchmarkový režim, který je součástí aplikace. Jeho spuštění zajišťuje tlačítko *Benchmark* v uživatelském rozhraní. Po aktivaci jsou postupně spuštěny všechny implementované generovací metody pro pět předem definovaných velikostí map: 20×50 , 60×60 , 70×30 , 100×100 a 150×150 dlaždic. Tyto velikosti byly zvoleny tak, aby pokrývaly jak malé, tak velké rozměry map a zároveň umožňovaly posoudit vliv tvaru mapy (čtverec versus obdélník) na chování algoritmů.

Pro každou kombinaci algoritmu a velikosti mapy je generování provedeno opakovaně v deseti nezávislých bězích. Tím je zajištěn dostatečný vzorek pro statistické vyhodnocení, zejména pro posouzení průměrných hodnot a variability výsledků.

Měření času generování je realizováno pomocí funkce `Time.get_ticks_usec()`, která poskytuje mikrosekundovou přesnost. Měření je výhradně čas strávený ve funkci `generate(width, height)` příslušného algoritmu. Vizualizace mapy, vykreslování dlaždic, umísťování hráče ani další operace nejsou součástí měřeného intervalu. Výsledný čas je převeden na milisekundy.

Pro každý běh jsou zaznamenány následující metriky:

- **čas generování** (`gen_time_ms`) – doba trvání samotného algoritmu v milisekundách,
- **počet podlahových dlaždic** (`floor_tiles`) – celkový počet průchozích buněk ve výsledné mapě,
- **pokrytí** (`coverage`) – podíl průchozích dlaždic z celkového počtu buněk mapy, vyjádřený v procentech jako $\frac{\text{floor_tiles}}{\text{width} \times \text{height}} \times 100$.

Výsledky jednotlivých běhů jsou průběžně ukládány do souboru `results.csv`. Každý záznam obsahuje název algoritmu, rozměry mapy, pořadí běhu, čas generování, pokrytí a počet podlahových dlaždic.

Pro vyhodnocení naměřených dat byl vytvořen samostatný skript `result_analyser.py` v jazyce Python s využitím knihoven `pandas` a `matplotlib`. Skript načte soubor `results.csv`, odfiltruje neplatné záznamy a pro každou kombinaci algoritmu a velikosti mapy vypočítá souhrnné statistiky – průměr, směrodatnou odchylku, minimum a maximum času generování i pokrytí. Tyto statistiky jsou uloženy do souboru `results_summary.csv`. Následně skript automaticky vygeneruje sadu grafů: sloupcové grafy průměrného času a pokrytí pro jednotlivé algoritmy, škálovací křivky v závislosti na velikosti mapy, boxploty rozložení hodnot a bodový graf kompromisu mezi rychlostí a pokrytím. Všechny grafické výstupy použité v následující kapitole byly vygenerovány tímto skriptem.

4 Výsledky

Tato kapitola představuje výsledky experimentálního měření implementovaných generovacích metod. Nejprve jsou analyzovány vlastnosti jednotlivých algoritmů z hlediska času generování a pokrytí mapy průchozím prostorem. Následně jsou algoritmy vzájemně porovnány a výsledky shrnuty v souhrnné tabulce.

Všechny grafy v této kapitole byly vygenerovány z dat naměřených benchmarkovým režimem popsáním v předchozí podkapitole. Popisky os a nadpisy grafů jsou uvedeny v anglickém jazyce, v popiscích obrázků je uveden český překlad.

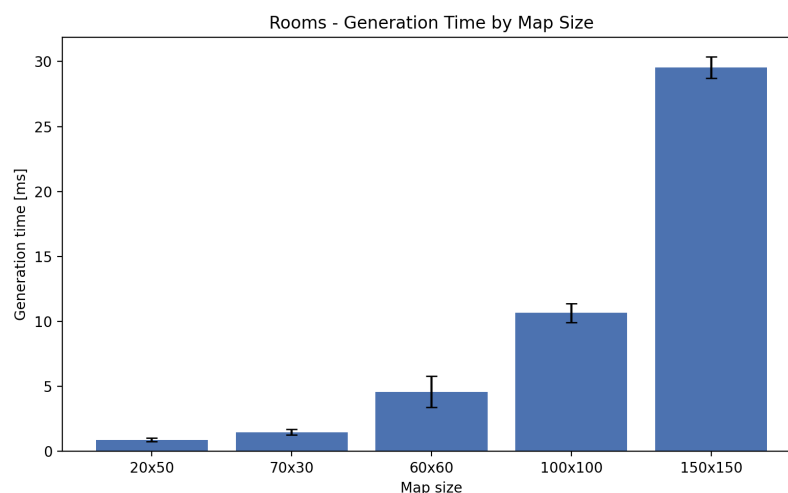
4.1 Výsledky jednotlivých algoritmů

4.1.1 Rooms (místnosti a chodby)

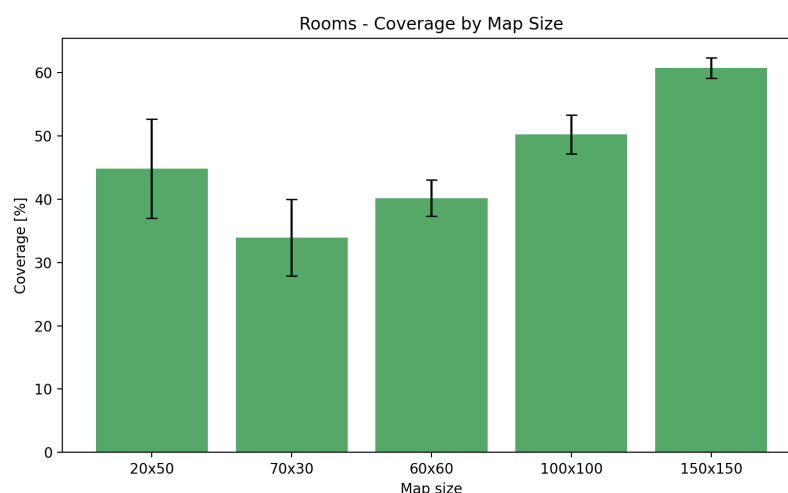
Algoritmus Rooms vykazuje nízké časy generování na menších mapách – na mapě 20×50 přibližně 0,89 ms. S rostoucí velikostí mapy čas narůstá výrazněji, protože počet generovaných místností je úměrný ploše mapy. Na mapě 150×150 dosahuje průměrný čas přibližně 29,5 ms.

Pokrytí mapy průchozím prostorem roste s velikostí mapy. Na menších mapách (20×50 , 70×30) se pohybuje v rozmezí 34–45 %, na mapě 100×100 činí přibližně 50 % a na mapě 150×150 dosahuje přibližně 61 %. Tento rostoucí trend je dán tím, že počet místností se odvíjí od celkové plochy mapy. Na větších mapách tak vzniká úměrně více místností, které efektivněji vyplňují dostupný prostor.

Variabilita mezi jednotlivými běhy je u tohoto algoritmu patrná jak v čase generování, tak v pokrytí. Pokrytí se mění podle velikosti a rozmístění náhodně vygenerovaných místností. Na menších mapách, kde je generován minimální počet pěti místností, je variabilita pokrytí vyšší.



Obrázek 7: Rooms – průměrný čas generování podle velikosti mapy (Generation Time by Map Size)



Obrázek 8: Rooms – průměrné pokrytí podle velikosti mapy (Coverage by Map Size)

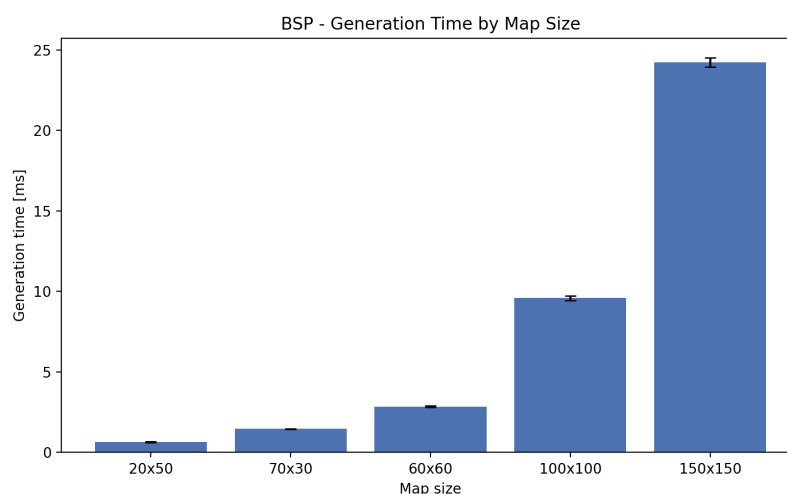
4.1.2 BSP (Binary Space Partitioning)

Algoritmus BSP vytváří mapy s dobře definovanými místnostmi a systematickým propojením. Čas generování roste s velikostí mapy – od přibližně 0,65 ms na mapě 20×50 po 24,2 ms na mapě 150×150. Tento nárůst odpovídá rekurzivní povaze algoritmu, který musí rozdělit celý prostor mapy a následně propojit vzniklé místnosti.

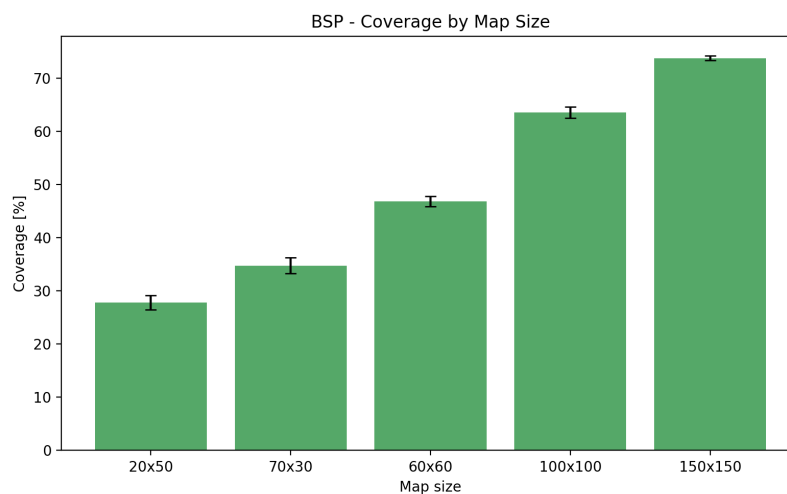
Pokrytí mapy u BSP roste s velikostí mapy. Na nejmenší mapě 20×50 činí pokrytí přibližně 27,8 %, zatímco na mapě 150×150 dosahuje hodnot kolem

73,8 %. Tento jev je dán principem algoritmu, který systematicky rozděluje celý dostupný prostor a v každé vzniklé oblasti vytváří místnost. Na větší mapě tak vzniká více a větších místností, které efektivněji vyplňují celkovou plochu.

Variabilita mezi běhy je u BSP relativně nízká. Směrodatné odchylky času i pokrytí jsou malé, což odpovídá determinističtější povaze rekurzivního dělení prostoru.



Obrázek 9: BSP – průměrný čas generování podle velikosti mapy (Generation Time by Map Size)



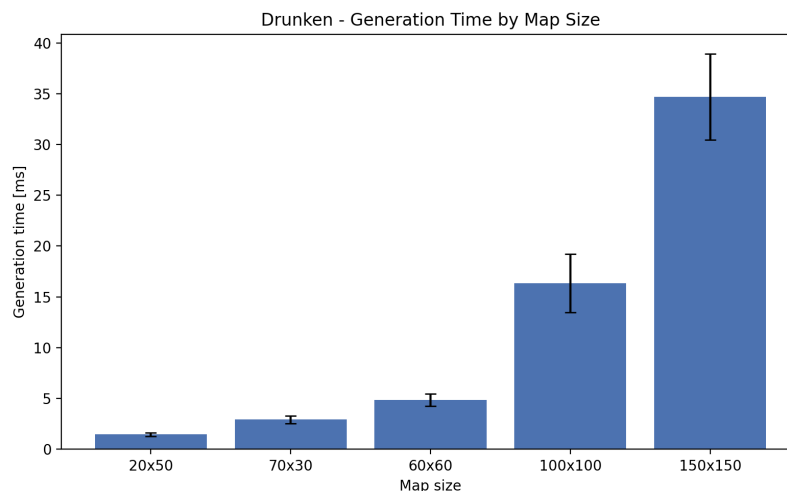
Obrázek 10: BSP – průměrné pokrytí podle velikosti mapy (Coverage by Map Size)

4.1.3 Drunken Walk (náhodná procházka)

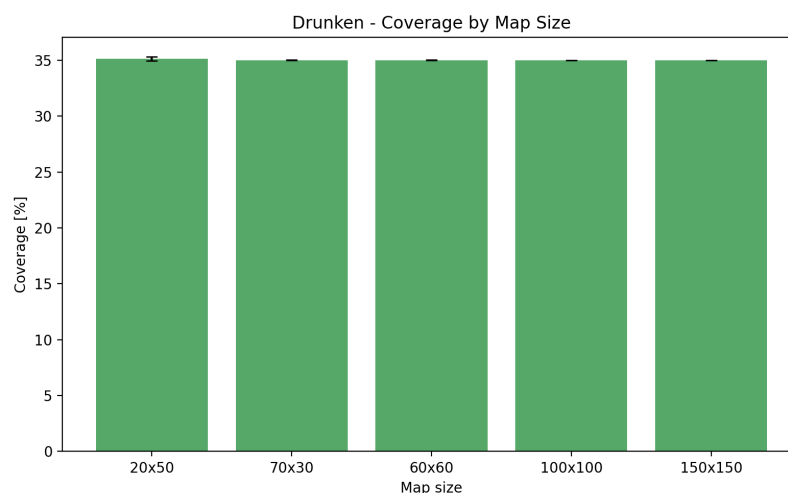
Algoritmus Drunken Walk se od ostatních metod odlišuje především konstantním pokrytím mapy. Výsledné pokrytí se ve všech testovaných konfiguracích pohybuje v rozmezí 35,0–35,1 %, což přímo odpovídá implementaci, ve které je cílový počet průchozích dlaždic stanoven jako pevný podíl celkové plochy mapy. Variabilita pokrytí je tedy prakticky nulová.

Čas generování naopak vykazuje výraznější variabilitu mezi jednotlivými běhy. Na mapě 20×50 se průměrný čas pohybuje kolem 1,5 ms, na největší mapě 150×150 dosahuje přibližně 34,7 ms. Rozptyl časů je způsoben stochastickou povahou algoritmu – náhodná procházka může v některých bězích rychle pokrýt nové oblasti, zatímco v jiných se opakovaně vrací do již odkrytých částí mapy. Hodnota 35 % je přitom nastavitelným parametrem implementace (konstanta v kódu), nikoliv inherentní vlastností algoritmu – změnou této konstanty lze dosáhnout libovolného cílového pokrytí.

Drunken Walk tak představuje zajímavý kontrast k buněčným automatům: zatímco Drunken Walk má předvídatelné pokrytí, ale nepředvídatelný čas (protože procházka může opakovaně navštěvovat již odkryté dlaždice), buněčné automaty mají naopak stabilní čas (vždy proběhne stejný počet simulačních kroků), avšak variabilní pokrytí závislé na náhodné počáteční konfiguraci.



Obrázek 11: Drunken Walk – průměrný čas generování podle velikosti mapy (Generation Time by Map Size)



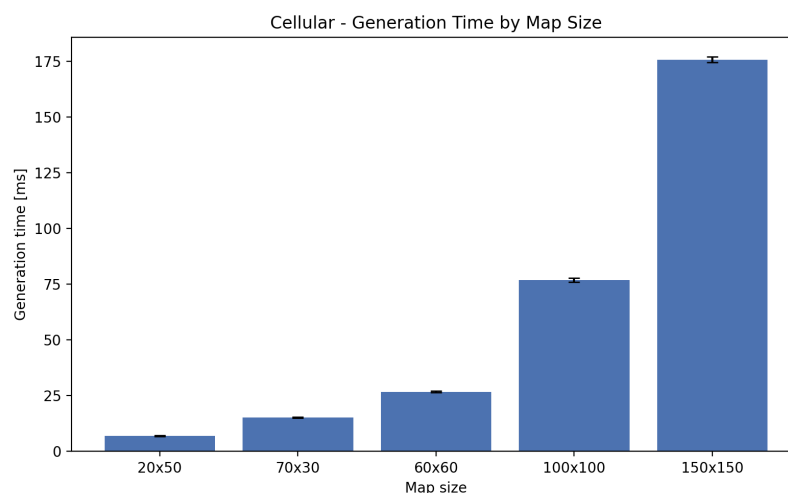
Obrázek 12: Drunken Walk – průměrné pokrytí podle velikosti mapy (Coverage by Map Size)

4.1.4 Cellular (buněčné automaty)

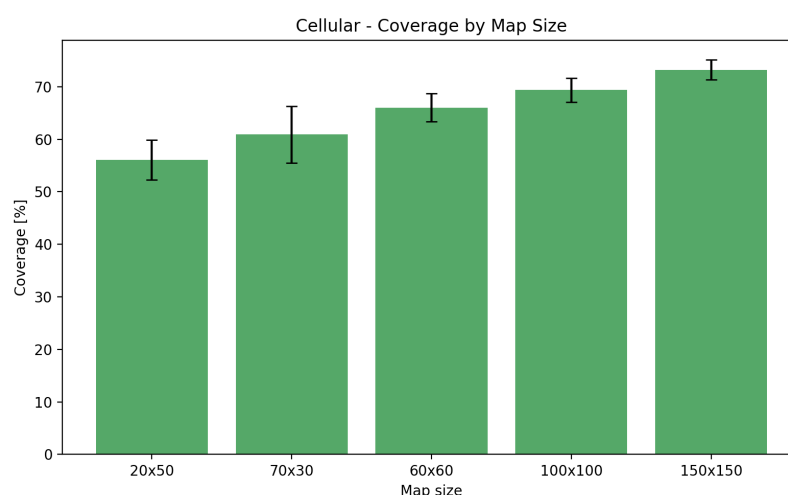
Buněčné automaty jsou z testovaných algoritmů jednoznačně nejpomalejší. Na nejmenší mapě 20×50 trvá generování přibližně 7,0 ms, na mapě 100×100 již 76,8 ms a na největší mapě 150×150 dosahuje průměrný čas přibližně 175,7 ms. Tento výrazný nárůst je způsoben tím, že algoritmus v každém simulačním kroku prochází všechny buňky mapy a vyhodnocuje jejich sousedství.

Pokrytí mapy u buněčných automatů dosahuje vysokých hodnot a s rostoucí velikostí mapy mírně roste – od přibližně 56 % na mapě 20×50 po 73,2 % na mapě 150×150 . Variabilita pokrytí je přitom nezanedbatelná, zejména na menších mapách, kde náhodná inicializace má větší vliv na výsledný tvar průchozích oblastí.

Čas generování je mezi běhy relativně stabilní, směrodatné odchylky jsou malé. Hlavním faktorem ovlivňujícím dobu generování je velikost mapy.



Obrázek 13: Cellular – průměrný čas generování podle velikosti mapy (Generation Time by Map Size)



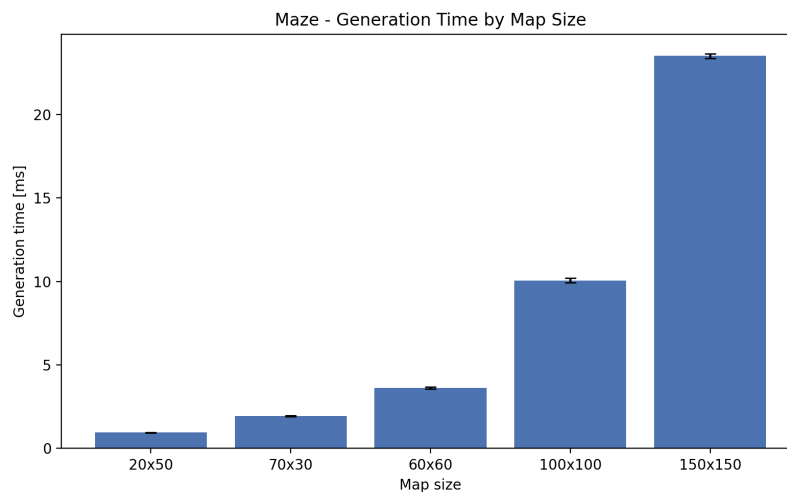
Obrázek 14: Cellular – průměrné pokrytí podle velikosti mapy (Coverage by Map Size)

4.1.5 Maze (bludiště)

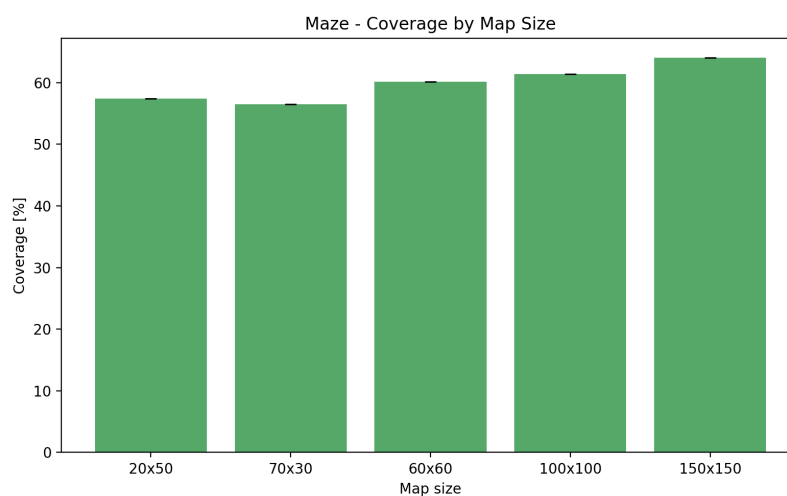
Algoritmus generování bludiště se vyznačuje zcela deterministickým pokrytím mapy. Hodnota pokrytí je pro danou velikost mapy vždy shodná bez ohledu na konkrétní běh. Na mapě 20×50 činí pokrytí 57,4 %, na mapě 150×150 pak 64,0 %. Nulová variabilita pokrytí vyplývá z principu algoritmu – *recursive backtracker* vždy navštíví všechny buňky mřížky a výsledný poměr průchozích dlaždic je dán pouze rozměry mapy a velikostí buněk.

Čas generování je u tohoto algoritmu srovnatelný s BSP. Na nejmenší mapě

činí přibližně 0,96 ms, na největší pak 23,5 ms. Variabilita času mezi běhy je velmi nízká, protože algoritmus vždy provede stejný počet kroků.



Obrázek 15: Maze – průměrný čas generování podle velikosti mapy (Generation Time by Map Size)



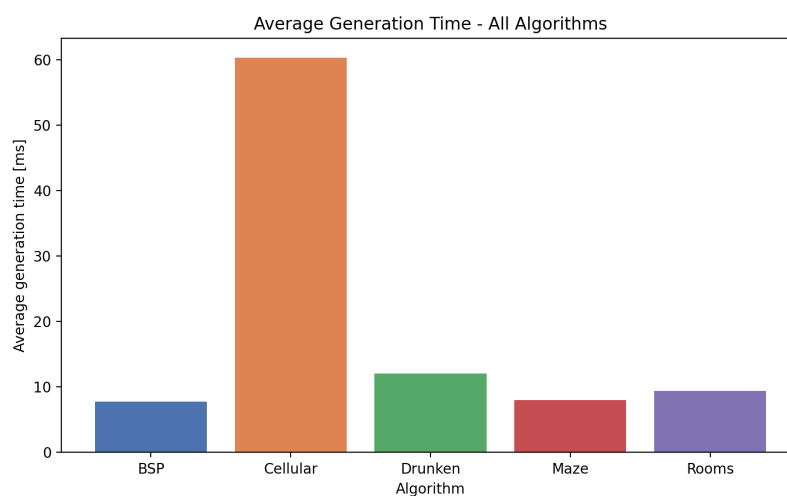
Obrázek 16: Maze – průměrné pokrytí podle velikosti mapy (Coverage by Map Size)

4.2 Srovnání algoritmů

Tato podkapitola porovnává všechny implementované metody z hlediska času generování, pokrytí mapy a jejich vzájemného vztahu.

4.2.1 Průměrný čas generování

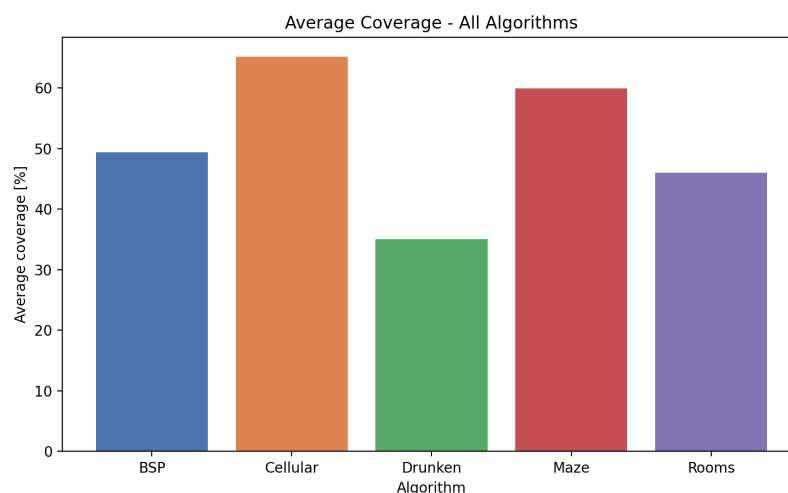
Celkový pohled na průměrné časy generování ukazuje výrazné rozdíly mezi jednotlivými metodami. Nejrychlejšími algoritmy jsou BSP a Maze, jejichž průměrné časy na mapě 150×150 dosahují přibližně 24 ms. Rooms se po úpravě algoritmu řadí do podobného rozmezí s časem přibližně 29,5 ms na největší mapě. Drunken Walk dosahuje na této velikosti přibližně 34,7 ms. Buněčné automaty jsou řádově nejpomalejší – na mapě 150×150 je jejich průměrný čas přibližně 175,7 ms, tedy přibližně sedmkrát více než u BSP a Maze.



Obrázek 17: Srovnání průměrného času generování všech algoritmů (Average Generation Time – All Algorithms)

4.2.2 Průměrné pokrytí

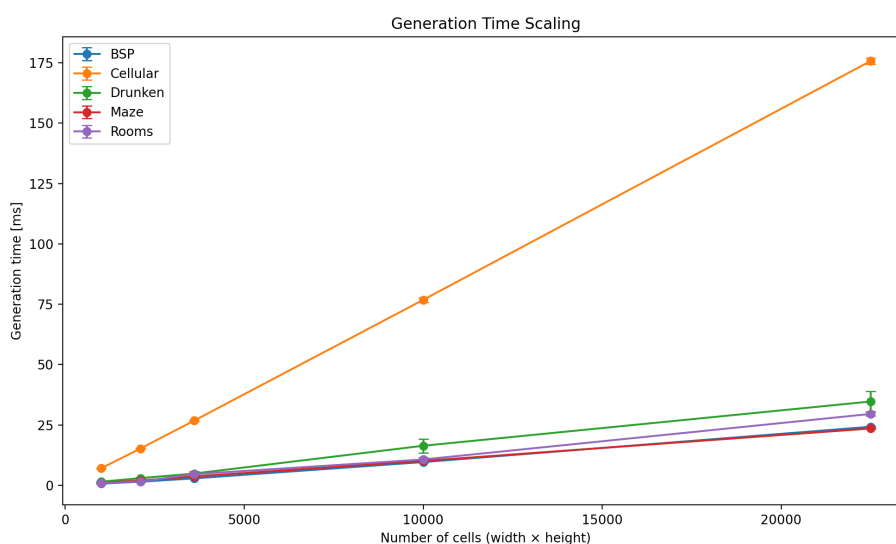
Z hlediska celkového průměrného pokrytí dosahují nejvyšších hodnot buněčné automaty a BSP, následované algoritmem Rooms. Maze dosahuje stabilního pokrytí kolem 60 %. Drunken Walk má konstrukčně dané pokrytí přibližně 35 %.



Obrázek 18: Srovnání průměrného pokrytí všech algoritmů (Average Coverage – All Algorithms)

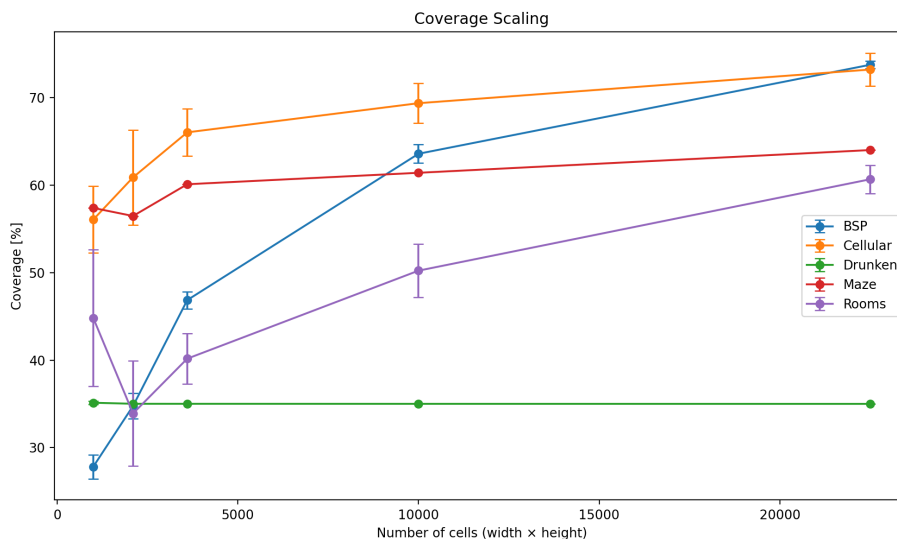
4.2.3 Škálování s velikostí mapy

Obrázek 19 znázorňuje, jak se čas generování mění s rostoucím počtem buněk mapy. BSP a Maze vykazují přibližně lineární nárůst času. Rooms roste rovněž přibližně lineárně, avšak s vyšší konstantou, protože na velkých mapách generuje výrazně více místností. Drunken Walk roste rychleji, a to kvůli opakovanému navracení do již odkrytých oblastí. Buněčné automaty vykazují nejstrmější nárůst, protože v každém simulačním kroku je nutné vyhodnotit všechny buňky mapy.



Obrázek 19: Škálování času generování s počtem buněk mapy (Generation Time Scaling)

Obrázek 20 ukazuje vývoj pokrytí v závislosti na velikosti mapy. BSP, Cellular a Rooms vykazují rostoucí tendenci – s větší mapou roste i podíl průchozího prostoru. U Rooms je tento nárůst dán škálováním počtu místností s plochou mapy. Maze má mírně rostoucí, ale velmi stabilní pokrytí. Drunken Walk zůstává konstantní na hodnotě přibližně 35 %.

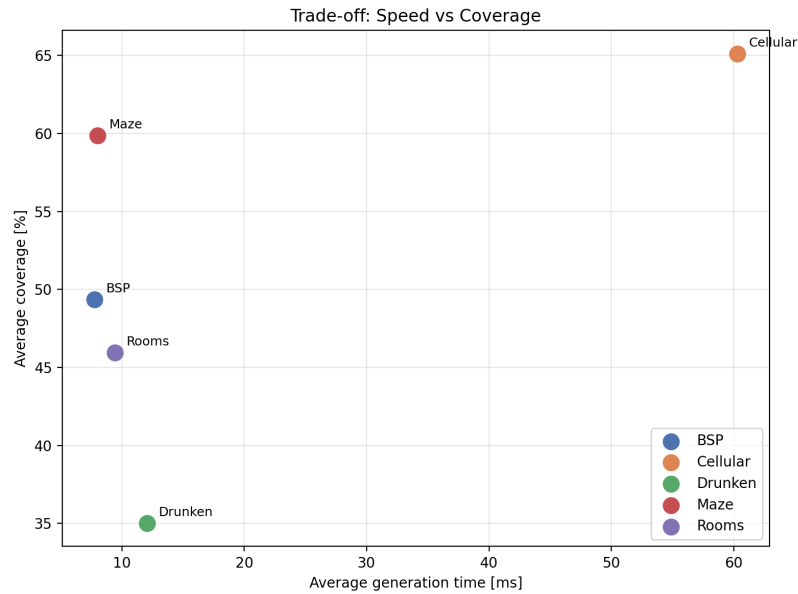


Obrázek 20: Škálování pokrytí s počtem buněk mapy (Coverage Scaling)

4.2.4 Kompromis mezi rychlostí a pokrytím

Obrázek 21 vizualizuje vztah mezi průměrným časem generování a průměrným pokrytím pro jednotlivé algoritmy. Ideální algoritmus by se nacházel v levé horní části grafu, tedy s nízkým časem a vysokým pokrytím.

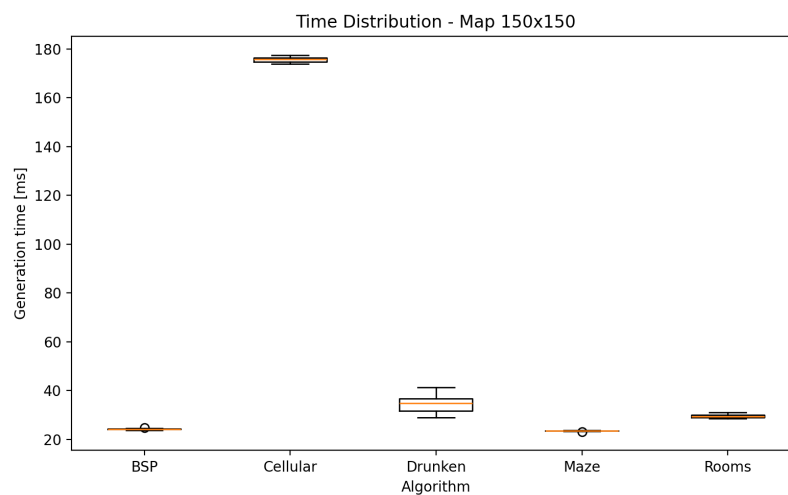
BSP a Maze se nacházejí v oblasti nízkého času a středně vysokého pokrytí. Rooms se po úpravě algoritmu posunul do oblasti vyššího pokrytí za cenu mírně vyššího času. Buněčné automaty dosahují nejvyššího pokrytí, avšak za cenu výrazně vyššího času. Drunken Walk představuje metodu s konstrukčně daným nižším pokrytím.



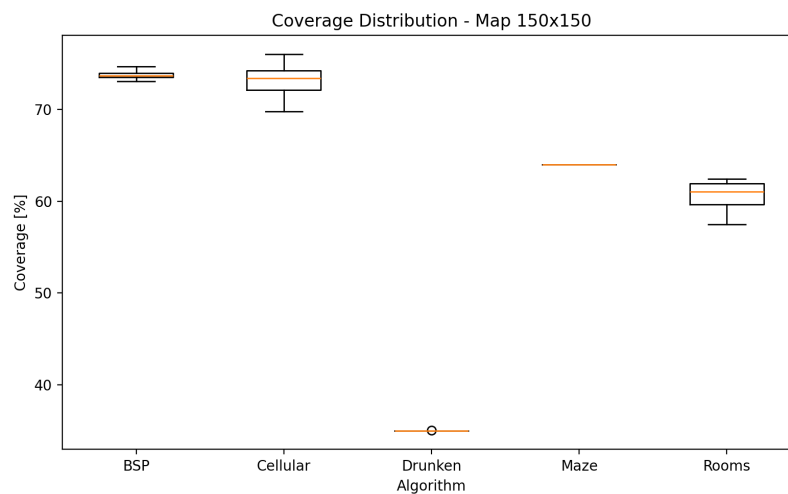
Obrázek 21: Kompromis mezi rychlostí a pokrytím (Trade-off: Speed vs Coverage)

4.2.5 Rozptyl výsledků na velké mapě

Pro detailnější pohled na variabilitu výsledků jsou uvedeny boxploty času generování a pokrytí na největší testované mapě 150×150 . Na obrázku 22 je patrné, že buněčné automaty vykazují nejmenší relativní rozptyl času (při výrazně vyšším absolutním čase), zatímco Drunken Walk má vyšší variabilitu. Obrázek 23 ukazuje, že Maze a Drunken Walk mají prakticky nulovou variabilitu pokrytí, zatímco Rooms, BSP a Cellular vykazují určitý rozptyl.



Obrázek 22: Rozptyl času generování na mapě 150×150 (Time Distribution – Map 150×150)



Obrázek 23: Rozptyl pokrytí na mapě 150×150 (Coverage Distribution – Map 150×150)

4.3 Souhrnná tabulka výsledků

Tabulka 3 shrnuje průměrné hodnoty času generování a pokrytí mapy pro všechny testované kombinace algoritmů a velikostí map.

Algoritmus	20×50	60×60	70×30	100×100	150×150
<i>Průměrný čas generování [ms]</i>					
Rooms	0,89	4,58	1,49	10,67	29,54
BSP	0,65	2,84	1,46	9,59	24,23
Drunken	1,47	4,84	2,92	16,35	34,69
Cellular	6,99	26,76	15,20	76,80	175,74
Maze	0,96	3,64	1,95	10,06	23,51
<i>Průměrné pokrytí [%]</i>					
Rooms	44,82	40,17	33,91	50,23	60,68
BSP	27,82	46,86	34,76	63,60	73,77
Drunken	35,13	35,01	35,01	35,01	35,00
Cellular	56,07	66,04	60,90	69,38	73,23
Maze	57,40	60,11	56,48	61,42	64,02

Tabulka 3: Souhrnné výsledky měření – průměrný čas generování a pokrytí

4.4 Shrnutí výsledků

Na základě provedených měření lze algoritmy rozdělit do dvou skupin podle výpočetní náročnosti. Do první skupiny patří BSP, Maze, Rooms a Drunken Walk, jejichž časy generování na mapě 150×150 nepřekračují 35 ms. Buněčné automaty tvoří samostatnou kategorii s průměrným časem přibližně 175,7 ms na této velikosti.

Z hlediska pokrytí mapy se algoritmy rovněž výrazně liší. BSP, Cellular a Rooms dosahují na velkých mapách pokrytí přes 60 %, přičemž BSP a Cellular překračují 73 %. Maze generuje stabilní pokrytí kolem 60–64 %. Drunken Walk má konstrukčně dané pokrytí přibližně 35 %.

5 Diskuse

5.1 Silné a slabé stránky jednotlivých metod

Na základě naměřených výsledků a zkušeností z implementace lze u každého algoritmu identifikovat specifické silné a slabé stránky, které ovlivňují jeho vhodnost pro různé typy her.

Algoritmus BSP nabízí dobrou rovnováhu mezi rychlostí generování, pokrytím mapy a strukturovaností výsledku. Generované mapy obsahují jasně oddělené místnosti propojené chodbami, což odpovídá tradičnímu pojetí dungeonových prostředí. Nevýhodou je poměrně pravidelný vzhled výsledných map, který může působit uniformně.

Rooms po úpravě škálování počtu místností dosahuje na velkých mapách výrazně vyššího pokrytí než bez této úpravy a zároveň nabízí přirozenější rozložení prostoru díky náhodnému umístování. Variabilita mezi běhy je u tohoto algoritmu vyšší, což zvyšuje znovuhratelnost, ale zároveň snižuje předvídatelnost výsledku.

Buněčné automaty generují přirozeně působící jeskynní struktury, které se nejvíce odlišují od klasického dungeonového prostředí. Dosahují nejvyššího pokrytí ze všech testovaných metod. Hlavní nevýhodou je výrazně vyšší výpočetní náročnost a možný vznik izolovaných oblastí, které nejsou vzájemně průchozí.

Drunken Walk vytváří organicky tvarované chodby s předvídatelným pokrytím. Konstantní pokrytí může být výhodou v situacích, kdy je požadována přesná kontrola nad poměrem průchozího prostoru. Nevýhodou je nižší strukturovanost výsledného prostředí – vznikají spíše propojené chodby než oddělené místnosti.

Algoritmus Maze byl do implementace zahrnut primárně pro demonstraci odlišného přístupu k procedurálnímu generování. Generuje dokonalé bludiště s nulovou variabilitou pokrytí a velmi nízkou variabilitou času. Z hlediska dungeonových map je však méně vhodný, protože vytváří převážně úzké chodby bez větších otevřených prostor.

5.2 Zkušenosti z implementace

Při implementaci buněčných automatů bylo nutné řešit problém izolovaných průchozích oblastí. U ostatních metod je průchodnost mapy zajištěna již principem algoritmu – explicitní propojování místností u Rooms a BSP, návštěva všech buněk u Maze a spojitá procházka u Drunken Walk.

Úprava algoritmu Rooms, při které byl pevný počet místností nahrazen škálováním úměrným ploše mapy, vedla k výraznému zlepšení pokrytí na velkých

mapách. Tato zkušenost ukazuje, že i drobná změna parametrizace může zásadně ovlivnit výsledné vlastnosti generovaného prostředí.

5.3 Souvislost s předchozím výzkumem

Porovnáním efektivity PCG algoritmů pro generování 2D map se zabývala rovněž práce [8], která měřila čas generování a spotřebu výpočetních zdrojů kombinací algoritmů pro umísťování místností (Random Room Placement, BSP) a propojování chodbami (Random Point Connect, Drunkard's Walk, BSP Corridors). Oproti tomuto přístupu se předkládaná práce zaměřuje na pět samostatných generovacích metod (včetně buněčných automatů a generování bludiště) a jako metriku používá vedle času generování také pokrytí mapy průchozím prostorem. Obě práce se shodují v závěru, že BSP patří mezi nejefektivnější přístupy z hlediska času generování.

5.4 Možnosti dalšího rozšíření práce

Implementované algoritmy by bylo možné dále rozšířit několika směry. Prvním z nich je kombinace více metod v rámci jedné mapy, například vyplnění BSP místností buněčnými automaty pro dosažení přirozenějšího vzhledu. Dalším rozšířením by mohlo být přidání sémantických pravidel pro rozmístění herních prvků, například zajištění, aby klíčové předměty byly umístěny v dostatečné vzdálenosti od startu hráče. V neposlední řadě by bylo možné implementovat adaptivní generování, které by přizpůsobovalo parametry algoritmů na základě postupu hráče hrou.

Závěr

Cílem této bakalářské práce bylo představit, implementovat a porovnat vybrané metody procedurálního generování dungeonových map ve 2D hrách. Tento cíl byl splněn.

V teoretické části práce byly popsány základní principy procedurálního generování obsahu, jeho historický vývoj a přehled vybraných metod včetně jejich výhod a omezení. Na základě provedené rešerše bylo vybráno pět algoritmů reprezentujících různé přístupy ke generování map: náhodné umístění místností, Binary Space Partitioning, náhodná procházka (Drunken Walk), buněčné automaty a generování bludiště.

Všech pět algoritmů bylo implementováno v herním enginu Godot s využitím skriptovacího jazyka GDScript. Systém byl navržen modulárně s jednotným rozhraním, které umožňuje snadné přidávání dalších generovacích metod. Součástí aplikace je automatizovaný benchmarkový režim, který zajišťuje objektivní a opakovatelné měření výkonu jednotlivých algoritmů.

Experimentální měření bylo provedeno na pěti velikostech map v deseti nezávislých bězích pro každou konfiguraci. Výsledky ukázaly výrazné rozdíly mezi metodami. Algoritmy BSP a Maze se ukázaly jako nejrychlejší s časy kolem 24 ms na mapě 150×150. Algoritmus Rooms dosahuje po úpravě škálování počtu místností pokrytí přibližně 61 % na největší mapě. Buněčné automaty generují mapy s nejvyšším pokrytím (přes 73 %), avšak s výrazně vyšším výpočetním časem přibližně 176 ms. Drunken Walk poskytuje konstrukčně dané pokrytí přibližně 35 %.

Práce ukázala, že volba generovací metody závisí na konkrétních požadavcích hry. Pro klasické dungeonové prostředí jsou vhodné algoritmy BSP a Rooms, pro jeskynní struktury buněčné automaty a pro bludiště algoritmus Maze. Modulární architektura navrženého systému umožňuje tyto metody snadno kombinovat a rozšiřovat.

Mezi omezení práce patří testování výhradně v prostředí Godot s jazykem GDScript, měření na jednom hardwarovém zařízení a nezahrnutí metod založených na umělé inteligenci. Možnosti dalšího rozšíření jsou diskutovány v kapitole 5.

Conclusions

The aim of this bachelor's thesis was to present, implement, and compare selected methods of procedural dungeon map generation in 2D games. This goal has been achieved.

The theoretical part described the basic principles of procedural content generation, its historical development, and an overview of selected methods, including their advantages and limitations. Based on the literature review, five algorithms representing different approaches to map generation were selected: random room placement, Binary Space Partitioning, drunkard's walk, cellular automata, and maze generation.

All five algorithms were implemented in the Godot game engine using the GDScript scripting language. The system was designed in a modular fashion with a unified interface that allows easy addition of new generation methods. The application includes an automated benchmark mode that ensures objective and repeatable performance measurements.

Experimental measurements were conducted on five map sizes with ten independent runs per configuration. The results revealed significant differences between the methods. BSP and Maze proved to be the fastest algorithms, with generation times around 24 ms on a 150×150 map. The Rooms algorithm, after adjusting its room count scaling, achieves approximately 61 % coverage on the largest map. Cellular automata generate maps with the highest coverage (over 73 %), but at the cost of significantly higher computation time of approximately 176 ms. Drunkard's walk provides a structurally fixed coverage of approximately 35 %.

The thesis demonstrated that the choice of generation method depends on the specific requirements of the game. BSP and Rooms are suitable for classic dungeon environments, cellular automata for cave-like structures, and the Maze algorithm for labyrinthine layouts. The modular architecture of the designed system allows these methods to be easily combined and extended.

Limitations of this work include testing exclusively in the Godot environment with GDScript, measurements on a single hardware device, and the exclusion of AI-based generation methods. Possible extensions are discussed in Chapter 5.

A Ukázky zdrojového kódu

Tato příloha obsahuje implementace pěti generovacích algoritmů v jazyce GDScript. Všechny algoritmy sdílejí společnou reprezentaci mapy (dvourozměrné pole s hodnotami 0, 1, 2) a funkci pro obalení podlahových dlaždic stěnami.

A.1 Generátor místností a chodeb

```
1 extends RefCounted
2
3 const TILE_EMPTY: int = 0
4 const TILE_FLOOR: int = 1
5 const TILE_WALL: int = 2
6
7 static func generate(width: int, height: int) -> Array:
8     var rng := RandomNumberGenerator.new()
9     rng.randomize()
10
11     var map: Array = []
12     for y: int in range(height):
13         var row: Array = []
14         for x: int in range(width):
15             row.append(TILE_EMPTY)
16         map.append(row)
17
18     var rooms: Array[Rect2] = []
19     @warning_ignore("integer_division")
20     var room_count: int = maxi(5, (width * height) / 360)
21     var max_attempts: int = room_count * 10
22     var tries: int = 0
23
24     while rooms.size() < room_count and tries < max_attempts:
25         var w: int = rng.randi_range(6, 16)
26         var h: int = rng.randi_range(6, 16)
27
28         if width < w + 2 or height < h + 2:
29             tries += 1
30             continue
31
32         var x: int = rng.randi_range(1, width - w - 1)
33         var y: int = rng.randi_range(1, height - h - 1)
34         var room: Rect2 = Rect2(x, y, w, h)
35
36         var overlaps: bool = false
37         for other in rooms:
38             if room.grow(1).intersects(other):
39                 overlaps = true
40                 break
41
42         if not overlaps:
43             rooms.append(room)
44             for iy: int in range(y, y + h):
```

```

45     for ix: int in range(x, x + w):
46         (map[iy] as Array)[ix] = TILE_FLOOR
47
48     if rooms.size() > 1:
49         var prev_center: Vector2 = rooms[rooms.size() - 2].get_center
50         ()
51         var curr_center: Vector2 = room.get_center()
52         _carve_corridor(map, prev_center, curr_center,
53             width, height, rng)
54     tries += 1
55
56 _add_walls(map, width, height)
57 return map
58
59 static func _carve_corridor(map: Array, from: Vector2,
60     to: Vector2, width: int, height: int,
61     rng: RandomNumberGenerator,
62     corridor_width: int = 2) -> void:
63     var half: int = int(corridor_width * 0.5)
64     var fx: int = int(from.x)
65     var fy: int = int(from.y)
66     var tx: int = int(to.x)
67     var ty: int = int(to.y)
68
69     if rng.randf() < 0.5:
70         for cx: int in range(min(fx, tx), max(fx, tx) + 1):
71             for off: int in range(-half, half + 1):
72                 var cy: int = fy + off
73                 if _in_bounds(cx, cy, width, height):
74                     (map[cy] as Array)[cx] = TILE_FLOOR
75             for cy: int in range(min(fy, ty), max(fy, ty) + 1):
76                 for off: int in range(-half, half + 1):
77                     var cx: int = tx + off
78                     if _in_bounds(cx, cy, width, height):
79                         (map[cy] as Array)[cx] = TILE_FLOOR
80     else:
81         for cy: int in range(min(fy, ty), max(fy, ty) + 1):
82             for off: int in range(-half, half + 1):
83                 var cx: int = fx + off
84                 if _in_bounds(cx, cy, width, height):
85                     (map[cy] as Array)[cx] = TILE_FLOOR
86             for cx: int in range(min(fx, tx), max(fx, tx) + 1):
87                 for off: int in range(-half, half + 1):
88                     var cy: int = ty + off
89                     if _in_bounds(cx, cy, width, height):
90                         (map[cy] as Array)[cx] = TILE_FLOOR
91
92 static func _in_bounds(x: int, y: int,
93     width: int, height: int) -> bool:
94     return x >= 0 and y >= 0 and x < width and y < height

```

Zdrojový kód 1: Generátor místností a chodeb (rooms_generator.gd).

A.2 Generátor BSP

```
1 extends RefCounted
2
3 const TILE_EMPTY: int = 0
4 const TILE_FLOOR: int = 1
5 const TILE_WALL: int = 2
6
7 class Branch:
8     var left_child: Branch
9     var right_child: Branch
10    var position: Vector2i
11    var size: Vector2i
12    var padding: Vector4i
13
14    func _init(pos: Vector2i, sz: Vector2i,
15        rng: RandomNumberGenerator) -> void:
16        position = pos
17        size = sz
18        padding = Vector4i(
19            rng.randi_range(2, 3), rng.randi_range(2, 3),
20            rng.randi_range(2, 3), rng.randi_range(2, 3))
21
22    func get_leaves() -> Array:
23        if not (left_child and right_child):
24            return [self]
25        return left_child.get_leaves() + right_child.get_leaves()
26
27    func get_center() -> Vector2i:
28        @warning_ignore("integer_division")
29        return Vector2i(position.x + size.x / 2,
30            position.y + size.y / 2)
31
32    func split(remaining: int, paths: Array,
33        rng: RandomNumberGenerator) -> void:
34        var split_horizontal: bool = size.y >= size.x
35        var split_percent: float = rng.randf_range(0.35, 0.65)
36
37        if split_horizontal:
38            var left_height: int = int(size.y * split_percent)
39            left_child = Branch.new(position,
40                Vector2i(size.x, left_height), rng)
41            right_child = Branch.new(
42                Vector2i(position.x, position.y + left_height),
43                Vector2i(size.x, size.y - left_height), rng)
44        else:
45            var left_width: int = int(size.x * split_percent)
46            left_child = Branch.new(position,
47                Vector2i(left_width, size.y), rng)
48            right_child = Branch.new(
49                Vector2i(position.x + left_width, position.y),
50                Vector2i(size.x - left_width, size.y), rng)
51
```

```

52     paths.push_back({
53         "left": left_child.get_center(),
54         "right": right_child.get_center()})
55
56     if remaining > 0:
57         left_child.split(remaining - 1, paths, rng)
58         right_child.split(remaining - 1, paths, rng)
59
60 static func generate(width: int, height: int) -> Array:
61     var map: Array = []
62     for y: int in range(height):
63         var row: Array = []
64         for x: int in range(width):
65             row.append(TILE_EMPTY)
66         map.append(row)
67
68     var rng := RandomNumberGenerator.new()
69     rng.randomize()
70
71     var paths: Array = []
72     var root_branch := Branch.new(
73         Vector2i(1, 1), Vector2i(width - 2, height - 2), rng)
74     root_branch.split(3, paths, rng)
75
76     for leaf in root_branch.get_leaves():
77         var pad: Vector4i = leaf.padding
78         for lx in range(leaf.size.x):
79             for ly in range(leaf.size.y):
80                 if _is_inside_padding(lx, ly, leaf, pad):
81                     continue
82                 var mx: int = lx + leaf.position.x
83                 var my: int = ly + leaf.position.y
84                 if mx > 0 and mx < width - 1 \
85                     and my > 0 and my < height - 1:
86                     (map[my] as Array)[mx] = TILE_FLOOR
87
88     for path in paths:
89         var a: Vector2i = path["left"]
90         var b: Vector2i = path["right"]
91         _carve_l_corridor(map, a, b, width, height)
92
93     _add_walls(map, width, height)
94     return map
95
96 static func _is_inside_padding(x: int, y: int,
97     leaf: Branch, pad: Vector4i) -> bool:
98     return x < pad.x or y < pad.y \
99         or x >= leaf.size.x - pad.z \
100         or y >= leaf.size.y - pad.w
101
102 static func _carve_l_corridor(map: Array,
103     a: Vector2i, b: Vector2i,
104     width: int, height: int) -> void:

```

```

105   for cx in range(min(a.x, b.x), max(a.x, b.x) + 1):
106       for dy in range(2):
107           var ty: int = a.y + dy
108           if cx > 0 and cx < width - 1 \
109               and ty > 0 and ty < height - 1:
110               (map[ty] as Array)[cx] = TILE_FLOOR
111   for cy in range(min(a.y, b.y), max(a.y, b.y) + 1):
112       for dx in range(2):
113           var tx: int = b.x + dx
114           if tx > 0 and tx < width - 1 \
115               and cy > 0 and cy < height - 1:
116               (map[cy] as Array)[tx] = TILE_FLOOR

```

Zdrojový kód 2: Generátor BSP (`bsp_generator.gd`).

A.3 Generátor náhodné procházky

```

1  extends RefCounted
2
3  const TILE_EMPTY: int = 0
4  const TILE_FLOOR: int = 1
5  const TILE_WALL: int = 2
6
7  static func generate(width: int, height: int) -> Array:
8      var map: Array = []
9      for y: int in range(height):
10         var row: Array = []
11         for x: int in range(width):
12             row.append(TILE_EMPTY)
13         map.append(row)
14
15     var rng: RandomNumberGenerator = RandomNumberGenerator.new()
16     rng.randomize()
17
18     @warning_ignore("integer_division")
19     var x: int = width / 2
20     @warning_ignore("integer_division")
21     var y: int = height / 2
22     (map[y] as Array)[x] = TILE_FLOOR
23
24     @warning_ignore("integer_division")
25     var target_floor_count: int = (width * height * 35) / 100
26     var current_floor_count: int = 1
27
28     current_floor_count += _carve_3x3(
29         map, x - 1, y - 1, width, height)
30     var steps_since_room: int = 0
31
32     while current_floor_count < target_floor_count:
33         var dir: int = rng.randi_range(0, 3)
34         match dir:
35             0: x += 1

```

```

36     1: x -= 1
37     2: y += 1
38     3: y -= 1
39
40     x = clamp(x, 2, width - 4)
41     y = clamp(y, 2, height - 4)
42     steps_since_room += 1
43
44     if steps_since_room > rng.randi_range(15, 25):
45         current_floor_count += _carve_3x3(
46             map, x - 1, y - 1, width, height)
47         steps_since_room = 0
48     else:
49         current_floor_count += _carve_2x2(
50             map, x, y, width, height)
51
52     _add_walls(map, width, height)
53     return map
54
55 static func _carve_3x3(map: Array, x: int, y: int,
56     width: int, height: int) -> int:
57     var carved_count: int = 0
58     for dy in range(3):
59         for dx in range(3):
60             var nx: int = x + dx
61             var ny: int = y + dy
62             if nx > 0 and nx < width - 1 \
63                 and ny > 0 and ny < height - 1:
64                 if (map[ny] as Array)[nx] != TILE_FLOOR:
65                     (map[ny] as Array)[nx] = TILE_FLOOR
66                     carved_count += 1
67     return carved_count
68
69 static func _carve_2x2(map: Array, x: int, y: int,
70     width: int, height: int) -> int:
71     var carved_count: int = 0
72     for dy in range(2):
73         for dx in range(2):
74             var nx: int = x + dx
75             var ny: int = y + dy
76             if nx > 0 and nx < width - 1 \
77                 and ny > 0 and ny < height - 1:
78                 if (map[ny] as Array)[nx] != TILE_FLOOR:
79                     (map[ny] as Array)[nx] = TILE_FLOOR
80                     carved_count += 1
81     return carved_count

```

Zdrojový kód 3: Generátor náhodné procházky (drunken_generator.gd).

A.4 Generátor buněčných automatů

```
1 extends RefCounted
```

```

2
3  const TILE_EMPTY: int = 0
4  const TILE_FLOOR: int = 1
5  const TILE_WALL: int = 2
6
7  const FILL_PROBABILITY: float = 0.45
8  const SIMULATION_STEPS: int = 5
9
10 static func generate(width: int, height: int) -> Array:
11     var rng := RandomNumberGenerator.new()
12     rng.randomize()
13
14     var map: Array = []
15     for y: int in range(height):
16         var row: Array = []
17         for x: int in range(width):
18             if x == 0 or x == width - 1 \
19                 or y == 0 or y == height - 1:
20                 row.append(TILE_EMPTY)
21             elif rng.randf() < FILL_PROBABILITY:
22                 row.append(TILE_FLOOR)
23             else:
24                 row.append(TILE_EMPTY)
25         map.append(row)
26
27     for _step in range(SIMULATION_STEPS):
28         map = _simulate_step(map, width, height)
29
30     _add_walls(map, width, height)
31     return map
32
33 static func _simulate_step(map: Array,
34     width: int, height: int) -> Array:
35     var new_map: Array = []
36     for y: int in range(height):
37         var row: Array = []
38         for x: int in range(width):
39             row.append(TILE_EMPTY)
40         new_map.append(row)
41
42     for y: int in range(height):
43         for x: int in range(width):
44             if x == 0 or x == width - 1 \
45                 or y == 0 or y == height - 1:
46                 (new_map[y] as Array)[x] = TILE_EMPTY
47                 continue
48
49         var floor_count: int = _count_floor_neighbours(
50             map, x, y, width, height)
51         if floor_count >= 4:
52             (new_map[y] as Array)[x] = TILE_FLOOR
53         else:
54             (new_map[y] as Array)[x] = TILE_EMPTY

```

```

55     return new_map
56
57 static func _count_floor_neighbours(map: Array,
58     x: int, y: int,
59     width: int, height: int) -> int:
60     var count: int = 0
61     for dy: int in range(-1, 2):
62         for dx: int in range(-1, 2):
63             if dx == 0 and dy == 0:
64                 continue
65             var nx: int = x + dx
66             var ny: int = y + dy
67             if nx >= 0 and nx < width \
68                 and ny >= 0 and ny < height:
69                 if (map[ny] as Array)[nx] == TILE_FLOOR:
70                     count += 1
71     return count

```

Zdrojový kód 4: Generátor buněčných automatů (cellular_generator.gd).

A.5 Generátor bludiště

```

1  extends RefCounted
2
3  const TILE_EMPTY: int = 0
4  const TILE_FLOOR: int = 1
5  const TILE_WALL: int = 2
6
7  const N: int = 1
8  const E: int = 2
9  const S: int = 4
10 const W: int = 8
11
12 static var cell_walls := {
13     Vector2i(0, -1): N, Vector2i(1, 0): E,
14     Vector2i(0, 1): S, Vector2i(-1, 0): W}
15
16 static var opposite_walls := {N: S, S: N, E: W, W: E}
17
18 static func generate(width: int, height: int) -> Array:
19     var map: Array = []
20     for y: int in range(height):
21         var row: Array = []
22         for x: int in range(width):
23             row.append(TILE_EMPTY)
24         map.append(row)
25
26     @warning_ignore("integer_division")
27     var cells_w: int = max(1, (width - 2) / 3)
28     @warning_ignore("integer_division")
29     var cells_h: int = max(1, (height - 2) / 3)
30

```

```

31  var grid: Dictionary = {}
32  for cy in range(cells_h):
33      for cx in range(cells_w):
34          grid[Vector2i(cx, cy)] = N | E | S | W
35
36  var stack: Array[Vector2i] = []
37  var current_cell := Vector2i(0, 0)
38  var unvisited: Dictionary = grid.duplicate()
39  unvisited.erase(current_cell)
40
41  var rng := RandomNumberGenerator.new()
42  rng.randomize()
43
44  while unvisited.size() > 0:
45      var neighbours := _check_neighbours(
46          current_cell, unvisited, cells_w, cells_h)
47      if neighbours.size() > 0:
48          var next: Vector2i = neighbours[
49              rng.randi() % neighbours.size()]
50          stack.append(current_cell)
51          var dir: Vector2i = next - current_cell
52          grid[current_cell] = grid[current_cell] \
53              - cell_walls[dir]
54          grid[next] = grid[next] - cell_walls[-dir]
55          current_cell = next
56          unvisited.erase(current_cell)
57      elif stack.size() > 0:
58          current_cell = stack.pop_back()
59      else:
60          if unvisited.size() > 0:
61              current_cell = unvisited.keys()[0]
62              unvisited.erase(current_cell)
63
64  for cell_pos in grid.keys():
65      var cell_x: int = cell_pos.x
66      var cell_y: int = cell_pos.y
67      var walls: int = grid[cell_pos]
68      var tile_x: int = 1 + cell_x * 3
69      var tile_y: int = 1 + cell_y * 3
70
71      for dy in range(2):
72          for dx in range(2):
73              var tx: int = tile_x + dx
74              var ty: int = tile_y + dy
75              if tx >= 1 and tx < width - 1 \
76                  and ty >= 1 and ty < height - 1:
77                  (map[ty] as Array)[tx] = TILE_FLOOR
78
79      if not (walls & N):
80          for dx in range(2):
81              var tx: int = tile_x + dx
82              var ty: int = tile_y - 1
83              if tx >= 1 and tx < width - 1 \

```

```

84         and ty >= 1 and ty < height - 1:
85             (map[ty] as Array)[tx] = TILE_FLOOR
86     if not (walls & E):
87         for dy in range(2):
88             var tx: int = tile_x + 2
89             var ty: int = tile_y + dy
90             if tx >= 1 and tx < width - 1 \
91                 and ty >= 1 and ty < height - 1:
92                 (map[ty] as Array)[tx] = TILE_FLOOR
93     if not (walls & S):
94         for dx in range(2):
95             var tx: int = tile_x + dx
96             var ty: int = tile_y + 2
97             if tx >= 1 and tx < width - 1 \
98                 and ty >= 1 and ty < height - 1:
99                 (map[ty] as Array)[tx] = TILE_FLOOR
100     if not (walls & W):
101         for dy in range(2):
102             var tx: int = tile_x - 1
103             var ty: int = tile_y + dy
104             if tx >= 1 and tx < width - 1 \
105                 and ty >= 1 and ty < height - 1:
106                 (map[ty] as Array)[tx] = TILE_FLOOR
107
108     _add_walls(map, width, height)
109     return map
110
111 static func _check_neighbours(cell: Vector2i,
112     unvisited: Dictionary,
113     cells_w: int, cells_h: int) -> Array[Vector2i]:
114     var list: Array[Vector2i] = []
115     for dir in cell_walls.keys():
116         var neighbour: Vector2i = cell + dir
117         if neighbour.x >= 0 and neighbour.x < cells_w \
118             and neighbour.y >= 0 \
119             and neighbour.y < cells_h:
120             if unvisited.has(neighbour):
121                 list.append(neighbour)
122     return list

```

Zdrojový kód 5: Generátor bludiště (maze_generator.gd).

Funkce `_add_walls`, která obklopí všechny podlahové dlaždice stěnami, je ve všech generátorech totožná:

```

1 static func _add_walls(map: Array,
2     width: int, height: int) -> void:
3     for y: int in range(height):
4         for x: int in range(width):
5             if (map[y] as Array)[x] == TILE_FLOOR:
6                 for dy: int in range(-1, 2):
7                     for dx: int in range(-1, 2):
8                         var nx: int = x + dx
9                         var ny: int = y + dy

```

```

10         if nx >= 0 and nx < width \
11             and ny >= 0 and ny < height:
12             if (map[ny] as Array)[nx] == TILE_EMPTY:
13                 (map[ny] as Array)[nx] = TILE_WALL

```

Zdrojový kód 6: Společná funkce pro přidání stěn.

B Odkaz na repozitář projektu

Zdrojový kód aplikace je dostupný v repozitáři na platformě GitHub:

<https://github.com/Jim-23/ProceduralGenerationBP>

C Zkratky

2D Two-Dimensional – dvourozměrný

3D Three-Dimensional – trojrozměrný

ASCII American Standard Code for Information Interchange – americký standardní kód pro výměnu informací

BSP Binary Space Partitioning – binární dělení prostoru

CSV Comma-Separated Values – hodnoty oddělené čárkou

DFS Depth-First Search – prohledávání do hloubky

PCG Procedural Content Generation – procedurální generování obsahu

RAM Random Access Memory – paměť s náhodným přístupem

D Obsah elektronických dat

Elektronická data práce jsou odevzdána ve formě ZIP archivu celého projektu s následující strukturou:

src/

Kompletní zdrojové soubory projektu a README.md s instrukcemi pro spuštění aplikace.

text/

Adresář s textem bakalářské práce.

Literatura

- [1] Shaker, Noor; Togelius, Julian; Nelson, Mark J. *Procedural Content Generation in Games: A Textbook and an Overview of Current Research*. 2016. 247 s. ISBN 978-3-319-42714-0.
- [2] Schuller, Dan. *Dive into beneath Apple Manor: A pre-rogue roguelike*. 2021. Dostupný z: <https://howtomakeanrpg.com/r/l/g/apple-manor.html>.
- [3] Wikipedia Contributors. *Beneath Apple Manor* — *Wikipedia, The Free Encyclopedia*. 2024. Dostupný z: https://en.wikipedia.org/w/index.php?title=Beneath_Apple_Manor&oldid=1260179812%22.
- [4] Procedural Content Generation Wiki. *Rogue* [online]. 2016 [cit. 2024-5-22]. Dostupný z: <http://pcg.wikidot.com/pcg-games:rogue>.
- [5] Wikipedia Contributors. *Rogue (video game)* — *Wikipedia, The Free Encyclopedia*. 2026. [Online; accessed 2026]. Dostupný z: [https://en.wikipedia.org/w/index.php?title=Rogue_\(video_game\)&oldid=1337873899](https://en.wikipedia.org/w/index.php?title=Rogue_(video_game)&oldid=1337873899).
- [6] Wikipedia Contributors. *Elite (video game)* — *Wikipedia, The Free Encyclopedia*. 2026. [Online; accessed 22-February-2026]. Dostupný z: [https://en.wikipedia.org/wiki/Elite_\(video_game\)](https://en.wikipedia.org/wiki/Elite_(video_game)).
- [7] Linden, Roland; Lopes, Ricardo; Bidarra, Rafael. Procedural Generation of Dungeons. *Computational Intelligence and AI in Games, IEEE Transactions on*. 2014, roč. 6, s. 78–89. Dostupný také z: <http://dx.doi.org/10.1109/TCIAIG.2013.2290371>.
- [8] Monaghan, Zina. *Comparing Procedural Content Generation Algorithms for Creating Levels in Video Games*. 2019. [Online; accessed 2026]. Dostupný také z: <https://arrow.tudublin.ie/cgi/viewcontent.cgi?article=1189&context=scschcomdis>.
- [9] *Godot Engine Documentation*. 2024. [Online; accessed 2026]. Dostupný z: <https://docs.godotengine.org>.
- [10] *GDScript Reference*. 2024. [Online; accessed 2026]. Dostupný z: <https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/>.
- [11] Pixel_Poem. *2D Pixel Dungeon Asset Pack*. 2023. [Online; accessed 2026]. Dostupný z: <https://pixel-poem.itch.io/dungeon-assetpack>.